

第 11 章: Assignments, Expressions, and Prints (赋值、表达式与输出)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 从第 11 章开始, 本书进入"逐条语句深入讲解"章节。第 8~10 章刚为你建立了 Python 语句语法(缩进、冒号、if、while)的初步印象, 本章则把赋值语句 (assignment)、表达式语句 (expression statement) 和 print 操作这三类最常用语句的细节一口气补齐——包括多重赋值、扩展解包、增强赋值、命名赋值表达式、print 的 file 参数与流重定向。它是后续所有章节的地基: for 循环的解包 (第 13 章)、match 模式 (第 12 章)、作用域 (第 17 章) 全都建立在本章的赋值语义之上。

11.1 开篇引言

英文原文

Now that we've had a first introduction to Python statement syntax, this chapter begins our in-depth tour of specific Python statements. We'll begin with the basics: assignment statements, expression statements, and print operations. We've already seen all of these

in action, but here we'll fill in important details we've skipped so far. Although they're relatively simple, as you'll see, there are optional variations for each of these statement types that will come in handy once you begin writing realistic Python programs.

中文翻译

既然我们已经初步了解了 Python 的语句语法，本章将开始对我们逐个深入研究 Python 的具体语句。我们先从基础讲起：赋值语句（assignment statement）、表达式语句（expression statement）和 print 操作（print operation）。本书前面已经让这些语句“跑起来”过，但这里我们要补齐之前刻意跳过的关键细节。虽然这些语句相对简单，但正如你将看到的，每一类语句都有一些可选的变体形式，一旦你开始编写真实的 Python 程序，它们就会派上大用场。

深度理解

- 核心概念：本章是“语句语法细节补全课”。重要性不在于让读者认识 `=` 或 `print`，而在于讲清它们的全部变体：普通赋值、元组/列表/序列赋值、带的扩展解包、多重目标赋值、增强赋值（`+=` 等）、赋值表达式（`:=`），以及 `print` 的四个关键字参数 `sep/end/file/flush` 和流重定向。
- 底层实现：Python 执行赋值时，本质是把等号右边表达式的对象引用（reference）存储到左边的名字里，全程不拷贝对象。这一点贯穿全章。
- 设计原因：Python 的赋值从不“创造变量声明”——名字在第一次赋值时才诞生。这带来一个关键安全特性：使用

未赋值/未声明的名字会直接抛异常，从而让拼写错误显形

。

·实际问题：这三类语句是任何 Python 程序里出现频率最高的语句，掌握细节能直接减少大量调试时间。

·初学者误区：以为"赋值 = 拷贝"，以为"变量是盒子"。正确心智是：变量是贴在对象上的便签（名字绑定对象），这在第 6 章已经埋下伏笔，本章将用它解释共享引用的一切诡异现象。

11.2 赋值总论 (Assignments)

英文原文

We've been using the Python assignment statement for a while to retain objects in examples. In its basic form, you write the target of an assignment on the left of an equals sign, and the object to be assigned on the right. The target on the left may be a name or object component, and the object on the right can be an arbitrary expression that creates an object. For the most part, assignments are straightforward, but here are a few key properties to note up front: Assignments create object references. As discussed in Chapter 6, Python assignments store references to objects in names or data structure components. They always create references to objects instead of copying the objects. Because of that, Python variables are more like poi

nters than data storage areas. Names are created when first assigned. Python creates a variable name the first time you assign it a value (i.e., an object reference), so there's no need to predeclare names ahead of time. Some (but not all) data structure slots are created when assigned, too (e.g., dictionary entries, some object attributes). Once assigned, a name is replaced with the value it references whenever it appears in an expression. Names must be assigned before being referenced. It's an error to use a name to which you haven't yet assigned a value. Python raises an exception if you try, rather than returning some sort of ambiguous default value. This turns out to be crucial in Python because names are not predeclared—if Python provided default values for unassigned names used in your program instead of treating them as errors, it would be much more difficult for you to spot name typos in your code. Some operations perform assignments implicitly. In this section, we're concerned with the `=` statement and its `:=` expression relative, but assignment occurs in many contexts in Python. For instance, you'll see later that module imports, function and class definitions, for loop variables, and function arguments are all implicit assignments. Because assignment works the same everywhere it pops up, all these contexts simply bind (i.e., assign) names and object components to object references at runtime. With those preliminaries in hand, let's move on to the code.

中文翻译

我们已经在示例中用 Python 赋值语句保存对象很多次了。它的基本形式是：把赋值的目标 (target) 写在等号左边，把要赋的对象 (object) 写在右边。左边的目标可以是一个名字或对象的组成部分，右边的对象可以是任何能创建对象的表达式。大体上，赋值是直截了当的，但有几个关键属性要提前说明：赋值创建对象引用 (object reference)。正如第 6 章所讨论的，Python 赋值是把对对象的引用存储到名字或数据结构组件中。它们总是创建对象的引用，而不是拷贝对象。正因为如此，Python 的变量更像指针 (pointer)，而不是数据存储区域。名字在首次赋值时才被创建。当你第一次给一个变量名赋一个值（即一个对象引用）时，Python 才创建这个名字，所以无需预先声明名字。有些（但不是全部）数据结构槽位也是在赋值时才创建的（例如字典条目、一些对象属性）。一旦赋值，每当这个名字出现在表达式中，它就会被替换为它引用的值。名字在被引用之前必须已被赋值。使用一个你尚未赋值的名字是错误。如果你尝试这样做，Python 会抛出异常，而不是返回某种含义模糊的默认值。这在 Python 中至关重要，因为名字不是预声明的——如果 Python 为程序中未赋值的名字提供默认值而不是把它们当作错误处理，你就很难发现代码中的拼写错误。有些操作会隐式地执行赋值。在本节中，我们关注的是 = 语句和它的表达式同类 :=，但赋值在 Python 中出现在许多场合。例如，你稍后会看到：模块导入 (module import)、函数与类定义、for 循环变量、函数参数都隐式地执行赋值。因为赋值在它出现的任何地方工作方式都一样，所有这些场合都只是把名字 (binding, 绑定) 和对象组件与

对象引用在运行时绑定起来。带着这些预备知识，让我们进入代码部分。

深度理解

·核心概念：赋值 = 把名字绑定到对象引用，四准则：①创建引用不拷贝；②名字即创造即存在；③无声明、用前必须赋值；④很多语句隐式赋值。

·底层实现：CPython 编译时，赋值语句编译为 STORENAME/STOREFAST/STORESUBSCR/STOREATTR 等字节码；引用一个未赋值名字时解释器在名字表（符号表）中找不到条目，于是抛 NameError。所谓“名字更像指针”，指名字本身只是指向堆中对象的一个槽位。

·设计原因：Python 坚持“没有默认值、用错即报错”的设计，是为了在动态语言中保留静态语言那一半的安全感——让笔误在运行第一秒就暴露，而不是潜伏成大坑。

·实际问题：理解“赋值不拷贝”后，才能解释为什么 `b = a`；`b.append(1)` 会同时改 `a`——这不是 BUG，是引用语义的正常结果。

·初学者误区：把整型也当“盒子”。`x = 5`；`y = x`；`x = 6` 后 `y` 仍是 5，很多人疑惑——其实整型对象不可变，`x = 6` 只是让 `x` 改绑到新对象，`y` 仍指向旧对象 5。用“便签”模型解释就毫无违和感。

11.3 赋值语法形式 (Assignment Syntax Forms)

英文原文

Although assignment is a general and pervasive concept in Python, in this chapter we are primarily interested in assignment statements, plus one limited expression. Table 11-1 illustrates the different syntax forms available for coding assignments in Python.

Table 11-1. Assignment statement and expression forms

Operation	Interpretation
<code>target = 'Hack'</code>	Basic assignment
<code>code, hack = 'py', 'PY'</code>	Tuple assignment
<code>[code, hack] = ['py', 'PY']</code>	List assignment
<code>a, b, c, d = 'hack'</code>	Sequence assignment
<code>a, b = 'hack'</code>	Extended-unpacking assignment
<code>code = hack = 'python'</code>	Multiple-target assignment
<code>code += 1, hack = 2</code>	Augmented assignments
<code>(python: = 3.12) + 0.01</code>	Named assignment expression

The basic form atop Table 11-1 is by far the most common: binding a single target (a name or data structure component) to a single object (an expression result). In fact, you could get all your work done with this form alone. The other table entries represent forms that are all optional, but that programmers often find convenient in practice: Tuple and list assignments. The second and third forms in the table are related. When you code a tuple or list on the left side of the `=`, Python pairs objects on the right side with targets on the left by position and assigns them from left to right. For example, in the second line of Table 11-1, both sides

es are tuples (sans parentheses), and the names `code` and `hack` are assigned `'py'` and `'PY'`, respectively. The left of the `=` is special syntax but the right is a real object, which is why this is called "unpacking" assignment—components on the right are unpacked into targets on the left.

Sequence assignment. Tuple and list assignments were later generalized into instances of what we now call sequence assignment—any sequence of targets can be assigned to any sequence (really, iterable) of values, and Python assigns the items one at a time by position. We can even mix and match the types of the sequences involved. The fourth line in Table 11-1, for example, pairs a tuple of names with a string of characters: `a` is assigned `'h'`, `b` is assigned `'a'`, and so on.

Despite this flexibility, the item on the left of the `=` is still a tuple or list of assignment targets.

Extended-unpacking assignment. An even later assignment form allows more flexibility in how we assign portions of a sequence—or other iterable—to a sequence of targets. The fifth line in Table 11-1, for example, matches `a` with the first character in the string on the right, and the starred name `b` with the rest: `a` is assigned `'h'`, and `b` is assigned `['a','c','k']`. This provides an alternative to assigning the results of slicing operations. Starred collectors like this have also somewhat usurped the term unpacking, though this is an artifact

t more historical than technical.

Multiple-target assignment. The sixth line in Table 11-1 shows the multiple-target form of assignment. In this form, Python assigns a reference to the same object (the object farthest to the right) to all the targets on the left. In the table, the names `code` and `hack` are both assigned references to the same string object, `'python'`. The effect is the same as if we had run `hack = 'python'` followed by `code = hack`, as `hack` evaluates to the original string object (i.e., not a separate copy of that object).

Augmented assignments. The second-to-last line in Table 11-1 is an example of augmented assignment—a shorthand that combines an expression and an assignment in a concise way. Saying `code += 1`, for example, has the same effect as `code = code + 1`, but the augmented form requires less typing and is generally quicker to run. In addition, if the target of the assignment is mutable, an augmented assignment may run even quicker by choosing an in-place update operation instead of an object copy. As you'll see, there is one augmented assignment statement for most binary expression operators in Python (even the unused `@!`).

Named assignment expression. New in Python 3.8, the `:=` operator allows you to code assignment as an expression, which returns the value it assigns to a name

. This expression can be nested in places where assignment statements don't work syntactically, and in common roles allows you to both assign a name and use its value in the same place in your code.

中文翻译

虽然赋值是 Python 中一个通用且无处不在的概念，但本章我们主要关注的是赋值语句，外加一个受限的表达式。表 11-1 展示了 Python 中可用的各种赋值语法形式。表 11-1：赋值语句与表达式形式（表格见上文英文原表，解释列含义——基本赋值 / 元组赋值 / 列表赋值 / 序列赋值 / 扩展解包赋值 / 多重目标赋值 / 增强赋值 / 命名赋值表达式）基本（basic）形式是表 11-1 中最常见的一种：把一个目标（名字或数据结构组件）绑定到单个对象（一个表达式的结果）。事实上，仅用这一种形式你就能完成所有工作。表中的其他条目代表的形式都是可选的，但程序员在实践中常常发现它们很方便：元组赋值和列表赋值（tuple and list assignment）。表中的第二、三种形式是相关的。当你在 `=` 左边写一个元组或列表时，Python 把右边的对象与左边的目标按位置——配对，然后从左到右依次赋值。例如表 11-1 的第二行，两边都是元组（省略了括号），名字 `code` 和 `hack` 分别被赋值为 `'py'` 和 `'PY'`。等号左边是特殊语法，而右边是真实对象，这正是它被称为“解包”（unpacking）赋值的原因——右边的组件被拆解包装到左边的目标中。序列赋值（sequence assignment）。元组和列表赋值后来被推广为我们现在所称的序列赋值的实例——任何目标序列都可以被赋值为任何数值序列（确切地说是可迭代对象 iterable），Python 每次按位置给一个条目赋值。甚至可

以混搭两边的序列类型。比如表 11-1 第四行，把一个名字元组与一个字符串配对：a 被赋为'h'，b 被赋为'a'，依此类推。尽管如此灵活，= 左边的条目仍然是一个由赋值目标构成的元组或列表。扩展解包赋值（extended-unpacking assignment）。一个更晚出现的赋值形式，允许在把序列的一部分——或其他可迭代对象——赋给目标序列时拥有更大灵活性。例如表 11-1 第五行，a 匹配右边字符串的第一个字符，带星号的名字 b 匹配其余部分：a 被赋为'h'，b 被赋为 ['a','c','k']。这提供了切片赋值结果的替代方案。这种带星号的收集者（collector）也在某种程度上“篡夺”了解包（unpacking）这一术语，不过这更多是历史原因而非技术原因。多重目标赋值（multiple-target assignment）。表 11-1 第六行展示了赋值的多重目标形式。在这种形式中，Python 把同一个对象（最右边的对象）的引用赋给左边的所有目标。表中，code 和 hack 都被赋予对同一个字符串对象'python' 的引用。效果等同于先执行 hack = 'python' 再执行 code = hack，因为 hack 求值结果是原来的字符串对象（即不是该对象的独立拷贝）。增强赋值（augmented assignment）。表 11-1 倒数第二行是增强赋值的例子——一种把表达式和赋值简洁地结合起来的简写。例如写 code += 1 的效果与 code = code + 1 相同，但增强形式打字更少、运行一般更快。此外，如果赋值的目标可变（mutable），增强赋值还可能通过选择原地（in-place）更新操作而不是对象拷贝而运行得更快。正如你将看到的，Python 中几乎所有二元表达式运算符都有一个对应的增强赋值语句（甚至包括没被用到的 @！）。命名赋值表达式（named assignment expression）。从 Python 3.8 起新增的 := 运算符，允许你把赋值写成表达式，它会把

赋给名字的值作为结果返回。这个表达式可以嵌套在赋值语句在语法上无法出现的地方，在常见用途中能让你在代码同一处既给名字赋值又使用它的值。

深度理解

·核心概念：一张表看懂本章全貌——八种赋值形式，从左下到右上复杂度递增：从"最基础"到"解包"再到"原地优化"再到"可嵌套的表达式化赋值"。每种形式都不是语法糖摆设，都有真实使用场景。

·底层实现：`=`恒等于把右边对象的引用塞进左边目标。元组/列表赋值本质上也是连续的单目标赋值（`code = 'py'; hack = 'PY'`）；`a, b = b, a` 交换变量是因为右边的 `(b, a)` 先求出为新元组，再解包。`+=` 编译为 `INPLACEADD` 类字节码（对可变对象走原地修改路径）。`:=` 编译为 `STOREFAST`（赋值副作用）+ `LOADFAST`（返回值），是一条表达式里的两次操作。

·设计原因：之所以引入这么多表单，是"让常见工作一行写完"的哲学：交换变量、取首尾、初始化计数器、读文件行——这些高频模式都应该有最优雅写法。

·实际问题：看别人代码时见到 `a, rest = ...`、`x = y = 0`、`cnt += 1`、`while line := f.readline():` 不再懵，并能判断什么时候该用哪种。

·初学者误区：把 `a, b = b, a` 理解为"同时交换"——其实是先求值右边再解包；把 `a = b = []` 理解为"创建了两个空列表"——其实只有一个列表被两个名字共享。这两大误区本章后面会反复敲打。

代码分析

```
table = {  
    "":          "target = 'Hack'",          # -> ...  
    "":          "code, hack = 'py', 'PY'",    # ...  
    "":          "[code, hack] = ['py', 'PY']", # ...  
    "":          "a, b, c, d = 'hack'",         # ...  
    "":          "a, *b = 'hack'",             # *b ...  
    "":          "code = hack = 'python'",      # ...  
    "":          "code += 1, hack *= 2",        # ...  
    "":          "(python := 3.12) + 0.01",     # := ...  
}
```

解释器如何处理：编译阶段，以上每种形式分别被编译成不同字节码组合：普通赋值 → STORE；元组/序列解包 → 结合 UNPACKSEQUENCE（在 Python 3.11+ 改为 UNPACKSEQUENCE 加 STORE 序列，带的用 UNPACKEX）；多重目标赋值 → 重复多次 DUPTOP + STORE（先算出唯一对象再重复存储到各名字）；+= → 先在目标上执行二元操作再存储（对列表等触发 INPLACE 魔法方法 iadd）。运行时"名字绑定对象"意味着左边名字从未被"复制对象"，只是表里新增一个指向该对象的条目。

11.4 基本赋值 (Basic Assignments)

英文原文

Let's turn to examples at the REPL prompt as usual. We've already used Table 11-1's basic assignment in this book, so you should be familiar with its basics. In short, it assigns a single target to a single value, wher

e the target may be a name, index, slice, or attribute, and the value is any expression: `$ python3 >>> L = [1, 2] # Name target >>> L[0] = 3 # Index target >>> L[-1:] = [4, 5] # Slice target >>> L [3, 4, 5] Attribute targets crop up for classes; we haven't studied these yet, but the assignment is straightforward: object.attr = L # Attribute target (see Part VI) Though names are common and dominate examples here, any of these four types of assignment targets can be used in any form of assignment, except where noted ahead (named assignment expressions, for example, allow only names). NOTE — Name annotations: Basic assignment statements can also have an annotation expression introduced by a colon immediately after the target on the left. This statement form allows only a single target, and is used for type hinting, described near the end of Chapter 6. As noted there, because this is optional, convoluted, completely unused by Python, and fundamentally at odds with the language's core idiom, we're skipping it in this book. See Python's docs for more details if you ever stumble in the wild onto the curious case of type declarations in a dynamically typed language.`

中文翻译

和往常一样，我们到 REPL 提示符下看例子。本书已经用过表 11-1 的基本赋值，所以你应该已经熟悉它的基础。简言之，它把一个目标赋给单个值，目标可以是名字、索引（in

dex)、切片(slice)或属性(attribute), 值可以是任何表达式: `$ python3 >>> L = [1, 2] # 名字目标>>> L[0] = 3 # 索引目标>>> L[-1:] = [4, 5] # 切片目标>>> L[3, 4, 5]` 属性目标在类中才会出现; 我们还没学类, 但赋值本身很简单: `object.attr = L` # 属性目标(见第六部分) 虽然在例子中以名字为主, 但这四类赋值目标可以用于任何形式的赋值, 除非前面有特别说明(例如命名赋值表达式只允许名字)。注意(NOTE)——名字注解(name annotation): 基本赋值语句还可以带一个注解表达式, 由目标后面紧跟的冒号引入。这种语句形式只允许单个目标, 用于类型提示(type hinting), 在第6章末尾有描述。正如那里所指出的, 由于它可选、绕弯、Python运行时完全不使用、而且与语言的核心习惯用法根本冲突, 本书选择跳过它。如果你在外面偶然遇到"动态类型语言里的类型声明"这种奇案, 请查阅Python官方文档了解详情。

深度理解

- 核心概念: 目标有四类——名字、索引、切片、属性。基本赋值 = 一个目标 ← 一个表达式结果。
- 底层实现: 四类目标对应四种存储字节码: STOREFAST/STORENAME(名字)、STORESUBSCR(索引, 调用对象的setitem)、STORESLICE在内部也归入下标操作、STOREATTR(属性, 调用setattr)。所以 `L[0] = 3` 实际上是在调用列表的方法来修改对象。
- 设计原因: Python把"名字/容器槽位/对象属性"统一都叫"目标", 好处是一个概念打天下——for循环变量、函数参数、解包目标全是同一种机制。

·实际问题：切片赋值 `L[-1:] = [4, 5]` 可以让列表长度变化（用 2 个元素替换 1 个槽位），这是“原地替换一段”的便捷手段，很多新手不知道它能改变长度。

·初学者误区：`L[0] = 3` 与 `L = [3,...]` 的区别——前者是修改既有列表对象，后者是让 `L` 改绑到新列表。前者影响所有共享该列表的其他名字，后者不影响。

·作者态度提示：名字注解 (`x: int = 3`) 是可选的、Python 运行时完全忽略的（仅存于 annotations），与动态类型语言核心哲学相悖，本书借用 Lutz 的立场刻意不教——但要知道它存在，读别人代码时不陌生。

代码分析

```
>>> L = [1, 2]           # L
>>> L[0] = 3             # 0
>>> L[-1:] = [4, 5]      # 1
>>> L
[3, 4, 5]
>>> object.attr = L      # __setattr__
```

解释器如何处理：第一步 INLIST 字面量求值后 STORENAME'L'；第二步先加载 `L`、压入下标 0 和值 3，执行 STORESUBSCR——它调用列表的 `setitem(L, 0, 3)` 原地修改；第三步 `L[-1:]` 仍是 STORESUBSCR，但 `setitem` 收到切片对象 `slice(-1, None, None)`，列表据此做“切片替换”；第四步属性赋值是 STOREATTR，触发 `setattr`。可见所有“赋值”都是对象协议（magic method）的调用，这正是 Python 一切皆是对象的体现。

11.5 序列赋值 (Sequence Assignments)

英文原文

Next up, here are a few simple and comparable examples of tuple and list assignment (a.k.a. sequence assignment) in action, unpacking items into individual variables:

```
>> first = 1 # Basic assignment >> second = 2 >> A  
, B = first, second # Tuple assignment >> A, B # Similar to A = first; B = second (1, 2) >> [C, D] = [first, second] # List assignment >> C, D (1, 2)
```

Notice that we really are coding two tuples in the third line in this interaction—we've just omitted their enclosing parentheses. Python pairs the values in the tuple on the right side of the assignment operator with the variables in the tuple on the left side and assigns the values one at a time. The same goes when `=` is surrounded by lists.

Tuple assignment leads to a common coding trick in Python that was introduced in a solution to the exercises at the end of Part II. Because Python creates a temporary tuple that saves the original values of the variables on the right while the statement runs, unpacking assignments are also an easy way to swap two variables' values without creating a temporary variable of your own—the tuple on the right remembers the pri

or values of the variables automatically:

```
>> first = 1 >> second = 2 >> first, second = second, first # Tuples: swaps values >> first, second # Like T = first; first = second; second = T (2, 1)
```

As already noted, the original tuple and list assignment forms in Python were eventually generalized to accept any type of sequence (really, iterable) on the right as long as it is of the same length as the sequence on the left. You can assign a tuple of values to a list of variables, a string of characters to a tuple of variables, and so on. In all cases, Python assigns items in the sequence on the right to targets in the sequence on the left by position—from left to right:

```
>> [a, b, c] = (1, 2, 3) # Assign tuple of values to list of names >> a, c (1, 3) >> (a, b, c) = 'ABC' # Assign string of characters to tuple names >> a, c ('A','C')
```

More broadly, while the left side of a sequence assignment is still a sequence (a tuple or list of targets), the right side may be any iterable object, not just any sequence. This is a more general category that includes collections both physical (e.g., lists) and virtual (e.g., a file's lines), which was first defined in Chapter 4 and has popped up in passing ever since.

We'll firm up this term when we explore iterables in Chapters 14 and 20, and apply it to unpack a range iterable in the next section. For now, the "sequence" in assignment is best associated with what's on the left

of the =.

Advanced sequence-assignment patterns Although we can mix and match sequence types around the = symbol, we must generally have the same number of items on the right as we have targets on the left, or we'll get an error. As you'll see in the next section, Python allows us to be more general with extended-unpacking syntax, but the number of items in the assignment target and subject must normally match:

```
>> string = 'TEXT' >> a, b, c, d = string # Same number on both sides
>> a, b, c, d = ('T', 'E', 'X', 'T') >> a, b, c = string # Error if not
ValueError: too many values to unpack (expected 3)
```

To be more flexible, we can always slice. There are a variety of ways to employ slicing to make this last case work:

```
>> a, b, c = string[0], string[1], string[2:] # Index and slice
>> a, b, c = ('T', 'E', 'XT') ('T', 'E', 'XT') >> a, b, c = list(string[:2]) + [string[2:]] # Slice and concatenate
>> a, b, c = ['T', 'E', 'XT'] ('T', 'E', 'XT') >> a, b = string[:2] # Slice and unpack directly
>> c = string[2:] # a, b = 'TE'; c = 'XT' >> a, b, c = ('T', 'E', 'XT') >> (a, b), c = string[:2], string[2:] # Nested sequences
>> a, b, c = (a, b), c = 'TE', 'XT' ('T', 'E', 'XT')
```

As the last example in this interaction demonstrates, we can even assign nested sequences, and Python unpacks their parts according to their shape, as expected.

d. In this case, we are assigning a tuple of two items, where the first item is a nested sequence (a string), exactly as though we had coded it this way:

```
>> ((a, b), c) = ('TE','XT') # Paired by shape and position
>> a, b, c ('T','E','XT')
```

Python pairs the first string on the right ('TE') with the first tuple on the left ((a, b)) and assigns one character at a time, before assigning the entire second string ('XT') to the variable c all at once. In this event, the sequence-nesting shape of the object on the left must match that of the object on the right. Nested sequence assignment like this is somewhat rare to see, but it can be convenient for picking out the parts of data structures with known shapes.

For example, you'll see in Chapter 13 that this technique also works in for loops, because loop items are assigned to the target given in the loop header just as if an = statement had been run:

```
for (a, b, c) in [(1, 2, 3), (4, 5, 6)]: ... # Simple tuple assignment
for ((a, b), c) in [((1, 2), 3), ((4, 5), 6)]: ... # Nested tuple assignment
```

Such nested sequence assignments can also be achieved with the match statement of the next chapter, but we'll hold back the details until then.

Sequence-unpacking assignments also give rise to another common coding idiom in Python—assigning an integer series to a set of variables:

```
>> red, green, blue = range(3) >> red, blue (0, 2)
```

This code initializes the three names on the left of `=` to the integers 0, 1, and 2, respectively. Because each name gets a unique value, it's Python's simplest equivalent to the enumerated data types you may have seen in other languages. To make sense of this, you need to recall that the `range` built-in function returns an iterable, which generates a list of successive integers (wrap it in a list if you wish to display its values at the REPL all at once like this):

```
>> list(range(3)) # list() required for display [0, 1, 2]
```

This call was previewed briefly in Chapter 4; because `range` is commonly used in `for` loops, we'll say more about it in Chapter 13. Another place you may see a tuple assignment at work is for splitting a sequence in to its front and the rest, in loops like the following's `while`, introduced in the prior chapter:

```
>> L = [1, 2, 3, 4] >> while L: # Repeat until L becomes empty (false)
```

```
... front, L = L[0], L[1:] # See next section for alternative ... print(front, L) ... 1 [2, 3, 4] 2 [3, 4] 3 [4] 4 []
```

The tuple assignment in the loop here could be coded as the following two lines instead, but it's often more convenient to string them together:

```
... front = L[0] ... L = L[1:] Notice that this code is using the list as a sort of stack data structure, which can
```

often also be achieved with the append and pop methods of list objects; here, `front = L.pop(0)` would have much the same effect as the tuple assignment statement, but it would be an in-place change. You'll learn more about while loops, and other (and often better) ways to step through a sequence with for loops, in Chapter 13.

中文翻译

接下来，是一些简单而可比较的元组与列表赋值（也就是序列赋值）实战例子，把条目解包到各个独立变量中：

```
>>> first = 1 # 基本赋值
>>> second = 2
>>> A, B = first, second # 元组赋值
>>> A, B # 相当于 A = first; B = second
(1, 2)
>>> [C, D] = [first, second] # 列表赋值
>>> C, D
(1, 2)
```

注意，这次的第三行里我们其实写了两个元组——只是省略了它们外面的括号。Python 把赋值运算符右边元组里的值与左边元组里的变量——配对，然后逐个赋值。当 = 两边是列表时也一样。元组赋值引出了 Python 中一个常见的小技巧，它在第二部分末尾练习题的解答中出现过。因为 Python 在语句运行期间会创建一个临时元组来保存右边变量的原值，所以解包赋值也是交换两个变量值而无需自己创建临时变量的简单方法——右边的元组自动记住了变量的先前值：

```
>>> first = 1
>>> second = 2
>>> first, second = second, first # 元组：交换值
>>> first, second
(2, 1)
```

如前所述，Python 中最初的元组和列表赋值形式最终被推广，只要右边的序列（确切地说可迭代对象 iterable）与左边的序列长度相同，就能接受任意类型的序列。你可以把一个

值元组赋给一个变量列表、把一个字符串赋给一个变量元组，等等。在所有情况下，Python 都按位置——从左到右——把右边序列中的条目赋给左边序列中的目标：

```
>>> [a, b, c] = (1, 2, 3) # 把值元组赋给名字列表>>> a, c = (1, 3)>>> (a, b, c) = 'ABC' # 把字符串赋给元组名字>>> a, c = ('A', 'C')
```

更广泛地说，虽然序列赋值左边仍是一个序列（由目标组成的元组或列表），右边可以是任何可迭代对象，而不只是任何序列。这是一类更宽泛的概念，既包括物理上的集合（如列表），也包括虚拟的集合（如文件的每一行），这个概念在第 4 章首次定义，此后一直穿插出现。我们将在第 14、20 章深入探讨可迭代对象时把这个术语敲定，并在下一节把它用于解包一个 range 可迭代对象。就目前而言，赋值中的“序列”最好与 = 左边的部分关联。高级序列赋值模式（advanced sequence-assignment patterns）虽然我们可以混搭 = 两边的序列类型，但通常右边条目的数量必须与左边的目标数量相同，否则会报错。正如你将在下一节看到的，扩展解包的语法允许我们更灵活，但赋值目标与数值主体中的条目数量通常必须匹配：

```
>>> string = 'TEXT'>>> a, b, c, d = string # 两边数量相同>>> a, b, c, d = ('T','E','X','T')>>> a, b, c = string # 数量不等则报错ValueError: too many values to unpack (expected 3)
```

要更灵活，我们总是可以切片。有各种运用切片让上面最后的例子成立的方法：

```
>>> a, b, c = string[0], string[1], string[2:] # 索引 + 切片>>> a, b, c = ('T','E','XT') ('T','E','XT')>>> a, b, c = list(string[:2]) + [string[2:]] # 切片 + 拼接>>> a, b, c = ['T','E','XT'] ('T','E','XT')>>> a, b = string[:2] # 切片后直接解包>>> c = string[2:] # a, b = 'TE'; c = 'XT'>>> a, b, c = ('T','E','XT')>>>
```

(a, b), c = string[:2], string[2:] # 嵌套序列>>> a, b, c #
(a, b), c = 'TE','XT'('T','E','XT') 正如这次交互中的最后一个例子所示，我们甚至可以对嵌套序列赋值，Python 会按照它们的形状 (shape) 如预期那样解包各部分。在这个例子里，我们赋的是一个两项元组，其中第一项本身是嵌套序列（一个字符串），完全就像我们这样写的一样：>>> ((a, b), c) = ('TE','XT') # 按形状和位置配对>>> a, b, c ('T','E','XT') Python 先把右边的第一个字符串 ('TE') 与左边的第一个元组 ((a, b)) 配对，逐个字符赋值，然后再把整个第二个字符串 ('XT') 一次性赋给变量 c。在这种情况下，左边对象的序列嵌套形状必须与右边对象的匹配。这种嵌套序列赋值比较少见，但对按已知形状从数据结构中挑出各部分来说很方便。例如，你将在第 13 章看到这种技巧同样适用于 for 循环，因为循环条目被赋给循环头中给出的目标，就像执行了一条 = 语句一样：for (a, b, c) in [(1, 2, 3), (4, 5, 6)]: ... # 简单元组赋值for ((a, b), c) in [((1, 2), 3), ((4, 5), 6)]: ... # 嵌套元组赋值这类嵌套序列赋值也可以借助下一章的 match 语句实现，但细节我们留到那时再讲。序列解包赋值还催生了 Python 中另一个常见编程习语——把一系列整数赋给一组变量：>>> red, green, blue = range(3) >>> red, blue (0, 2) 这段代码把左边 = 上的三个名字分别初始化为整数 0、1、2。由于每个名字都获得唯一的值，这就是 Python 对你在其他语言中见过的枚举 (enumerated) 数据类型的最简单的等价物。要理解它，你需要回忆起 range 这个内置函数返回一个可迭代对象 (iterable)，它生成连续整数序列（如果想在 REPL 里一次性显示它的值，就把它包进 list()）：>>> list(range(3)) # 显示需要用 list() 包裹[0, 1, 2] 这个调用在第 4 章简要预告过；由于 rang

e 常用于 for 循环，第 13 章还会有更多讲解。另一个你会看到元组赋值起作用的地方，是把一个序列切分成"头部"和"其余部分"，例如下面使用上一章引入的 while 的循环：>
>> L = [1, 2, 3, 4] >>> while L: # 重复直到 L 为空（逻辑假）... front, L = L[0], L[1:] # 下一节有的替代写法... print(front, L) ... 1 [2, 3, 4] 2 [3, 4] 3 [4] 4 [] 循环里的元组赋值也可以写成下面两行，但串成一行往往更方便：... front = L[0] ... L = L[1:] 注意这段代码把列表当作某种栈（stack）数据结构使用，这通常也可以用列表对象的 append 和 pop 方法实现；在这里 front = L.pop(0) 的效果与元组赋值语句非常相似，但它会做原地修改。关于 while 循环的更多内容，以及用 for 循环步进序列的其他（往往更好的）方式，你将在第 13 章学到。

深度理解

- 核心概念：序列赋值 = 按位置把右边可迭代对象逐项赋给左边目标；两边数量必须相等，否则 ValueError。核心特例：交换变量——右边先求值成临时元组。
- 底层实现：交换的秘诀在字节码顺序：a, b = b, a 先把 b 和 a 都加载到值为栈（LOADFAST），构成新元组，再 UNPACKSEQUENCE 拆开分别 STOREFAST 回 a、b。右值先于左值求值，所以"旧值"全部存活于临时元组中。嵌套解包的字节码是递归展开的 UNPACKSEQUENCE（Python 3.11 后为 UNPACKEX 处理）。
- 设计原因：为什么右边只要是 iterable 就行，而不强制是刺破？为了解包能作用于文件对象、range、字典等一切可迭代之物，实现"写给任何数据源的统一解包语法"

。这正是 Python "协议化"思想：迭代协议 (iter) 一统天下。

·实际问题：for front, rest in ..., a, b = b, a, red, green, blue = range(3) (颜色三元组初始化)、从函数返回多个值 return x, y 后 a, b = f() 接收——这些日常写法全是序列赋值的实战。

·初学者误区：①以为 a, b = b, a 是"奇技淫巧"——其实是标准语法，别的语言要写临时变量；②解包数目不齐报 ValueError: too many values to unpack (expected 3)，新手看这个报错不知道是解包数量问题；③把 range(3) 误当列表——它是懒生成的迭代器，只有迭代时才出数。

代码分析

```
>>> first, second = 1, 2          #
>>> first, second = second, first  #
>>> first, second                  # (2, 1) —
(2, 1)

>>> a, b, c, d = 'TEXT'           # a='T', b='E', c='X', d='T'
>>> a, b, c = string               # ValueError:
ValueError: too many values to unpack (expected 3)

>>> (a, b), c = ('TE', 'XT')      # a='T', b='E', c='XT'
>>> red, green, blue = range(3)    # red=0, green=1, blue=2

>>> L = [1, 2, 3, 4]
>>> while L:                        # while front L
...     front, L = L[0], L[1:]     # front = L[0]; L = L[1:]
...     print(front, L)            # 1 [2,3,4] / 2 [3,4] / 3 [4] /...
```

解释器如何处理：first, second = second, first 的字节码

流程大致为——LOADFAST second; LOADFAST first; ROTWO; STOREFAST first; STOREFAST second (3.11 前)，右边的两个旧值先被压进操作栈，所以即使先存了 first 也不会丢旧值。(a,b), c = ('TE','XT') 会先对元组 ('TE','XT') 执行拆包，检测到左边第一项也是容器，于是对 'TE' 再递归拆包——这就是"按形状解包"的底层含义。red, green, blue = range(3) 中 range 是迭代器类型的对象，解包协议逐次调用它的 next() 取得 0、1、2。while L 每一轮都会计算新的元组 (L[0], L[1:]) 再解包，从而让列表逐步变短直到为空。

11.6 扩展解包赋值 (Extended-Unpacking Assignments)

英文原文

The prior section demonstrated how to use manual slicing to make sequence assignments more general. In later Pythons, sequence assignment was further extended to make this easier.

In short, a starred target, X, can be used on the left of `f =` in order to specify a more general matching against the iterable on the right—the starred target is assigned a list, which collects all items in the iterable not assigned to other targets. This is especially handy for common coding patterns such as splitting a sequence into its "front" and "rest," as in the preceding exam

ple.

Extended unpacking in action Let's jump right into an example to demo how this works. As already shown, sequence assignments normally require exactly as many targets on the left as there are items in the object on the right. We get an error if the lengths disagree, unless we manually slice on the right, as shown in the prior section:

```
>> seq = [1, 2, 3, 4] >> a, b, c, d = seq >> a, d (1, 4)
>> a, b = seq ValueError: too many values to unpack
(expected 2)
```

We can, however, use a single starred target in this example to match more generally. In the following continuation of our interactive session, `a` matches the first item in the sequence, and `b` matches the rest:

```
>> a, b = seq >> a 1 >> b [2, 3, 4]
```

When a starred target is used (like name `b` here), the number of items on the left need not match the length of the iterable on the right (like sequence `seq` here). In fact, the starred target can appear anywhere on the left. For instance, in the next interaction `b` matches the last item in the sequence, and `a` matches everything before the last:

```
>> a, b = seq >> a [1, 2, 3] >> b 4
```

When the starred target appears in the middle, it collects everything between the other targets listed. Thu

s, in the following interaction, a and c are assigned the first and last items, and b gets everything in between them:

```
>> a, b, c = seq >> a 1 >> b [2, 3] >> c 4
```

More generally, wherever the starred target shows up, it will be assigned a list that collects every unassigned item at that position:

```
>> a, b, c = seq >> a 1 >> b 2 >> c [3, 4]
```

Naturally, like normal sequence assignment, extended-unpacking syntax works for any sequence types (really, again, any iterable), not just lists. Here it is unpacking characters in a string and an iterable range—which generates values 0...3 on demand:

```
>> a, b = 'hack' >> a, b ('h', ['a','c'],'k') >> a, b, c = 'hack' >> a, b, c ('h', ['a','c'],'k') >> a, b, c = range(4) >> a, b, c (0, [1, 2], 3)
```

This is similar in spirit to slicing, but not exactly the same—a sequence unpacking assignment always returns a list for matched items, whereas slicing returns a sequence of the same type as the object sliced:

```
>> S = 'hack' >> a, b, c = S[0], S[1:-1], S[-1] # Slices are type specific >> a, b, c ('h','ac','k') >> a, b, c = S # But always returns a list >> a, b, c ('h', ['a','c'],'k')
```

Finally, the closing example of the prior section becomes even simpler with this extension, since we don't have to manually slice to get the first and rest of the

items (though the "rest" L always becomes a list after the first):

```
>> L = [1, 2, 3, 4] >> while L:
```

```
... front, L = L # Get first, rest without slicing ... print(f
ront, L) ... 1 [2, 3, 4] 2 [3, 4] 3 [4] 4 []
```

Boundary cases Although extended unpacking is flexible, some boundary (atypical) cases are worth noting. First, the starred target may match just a single item, but is always assigned a list:

```
>> seq = [1, 2, 3, 4] >> a, b, c, d = seq >> print(a, b,
c, d) 1 2 3 [4]
```

Second, if there is nothing left to match the starred target, it is assigned an empty list, regardless of where it appears. In the following, names a, b, c, and d have matched every item in the sequence, but Python assigns e an empty list instead of treating this as an error case:

```
>> a, b, c, d, e = seq >> print(a, b, c, d, e) 1 2 3 4 []
>> a, b, e, c, d = seq >> print(a, b, c, d, e) 1 2 3 4 []
```

Errors can still be triggered, though, if there is more than one starred target, if there are too few values and no star (as before), and if the starred target is not itself coded inside a sequence:

```
>> a, b, c, d = seq SyntaxError: multiple starred expressions in assignment
>> a, b = seq ValueError: too many values to unpack (expected 2)
>> a = seq Synt
```

`axError: starred assignment target must be in a list or tuple`
`>> a, = seq # A one-item tuple sans parentheses`
`>> a # Same as a = seq, but makes a new copy! [1, 2, 3, 4]`

Technically, the single-appearance rule for starred targets on the left of `=` really applies only to each sequence when there is nesting on both sides—though you still can't have more than one per nested sequence:

```
>> [(a, b), (c, d)] = [(1, 2, 3, 4), (5, 6, 7, 8)] >> a, b, c, d
(1, [2, 3, 4], 5, [6, 7, 8]) >> [(a, b, x), (c, d)] = [(1, 2, 3, 4), (5, 6, 7, 8)]
SyntaxError: multiple starred expressions in assignment
```

And finally, any assignment target can be starred—though you may be hard-pressed to find some in real code:

```
>> a, L = [1, 2, 3] # Name target >> a, L (1, [2, 3]) >
> a, L[0] = [4, 5, 6] # Index target >> a, L (4, [[5, 6], 3])
>> a, L[1:] = [7, 8, 9] # Slice target >> a, L (7, [[5, 6], 8, 9])
>> class C:... # See Part VI >> a, C.attr = 'yikes'
# Attribute target >> a, C.attr ('y', ['i','k','e','s'])
```

A useful convenience Keep in mind that extended-unpacking assignment is just a convenience (well, in addition to a mouthful). We can usually achieve the same effects with explicit indexing and slicing, but extended unpacking is simpler to code. The common "first, rest" splitting coding pattern, for example, can be coded either way, but slicing is extra work:

```
>> seq [1, 2, 3, 4] >> a, b = seq # First, rest >> a, b (1, [2, 3, 4]) >> a, b = seq[0], seq[1:] # First, rest sans >> a, b (1, [2, 3, 4])
```

The also-common "rest, last" splitting pattern can similarly be coded either way, but extended-unpacking syntax again requires noticeably fewer keystrokes (and notably fewer neurons):

```
>> a, b = seq # Rest, last >> a, b ([1, 2, 3], 4) >> a, b = seq[:-1], seq[-1] # Rest, last sans >> a, b ([1, 2, 3], 4)
```

Being both simpler and arguably more natural, extended-unpacking syntax is also common in Python code.

Application to for loops Because the loop variable in the for loop statement can be any assignment target, extended unpacking works here too. We used the for loop iteration tool briefly in Chapter 4 and will study it formally in Chapter 13, but its tie-in here is straightforward: extended-unpacking assignments may show up after the word `for`, where a simple variable name is more commonly used:

```
for (a, b, c) in [(1, 2, 3, 4), (5, 6, 7, 8)]: ...
```

When used in this context, on each iteration Python simply assigns the next tuple of values to the tuple of targets. On the first loop, for example, it's as if we'd run the following assignment statement:

```
a, b, c = (1, 2, 3, 4) # b gets [2, 3]
```

The names `a`, `b`, and `c` can be used within the loop's code to reference the extracted components. In fact, this is really not a special case at all, but just an instance of general assignment at work. As we saw earlier in this chapter, for example, we can do similar in loops with simple sequence (tuple) assignment:

```
for (a, b, c) in [(1, 2, 3), (4, 5, 6)]: ... # a, b, c = (1, 2, 3), ...
```

And we can always emulate extended-unpacking assignment behavior by manually slicing:

```
for all in [(1, 2, 3, 4), (5, 6, 7, 8)]: ... a, b, c = all[0], all[1:-1], all[-1]
```

Since we haven't learned enough to get more detailed about the syntax of `for` loops, we'll put this topic on the back burner until its reprise in Chapter 13.

中文翻译

上一节演示了如何使用手动切片让序列赋值更灵活。在较新的 Python 中，序列赋值被进一步扩展，让这一切更容易。简言之，星号目标（starred target）`X` 可以出现在 `=` 左边，用来指定对右边可迭代对象更灵活的匹配——星号目标被赋成一个列表，收集可迭代对象中未被其他目标匹配的所有条目。这对于“把序列拆成头部与其余部分”这类常见编程模式特别方便，如前一个例子。扩展解包实战（extended unpacking in action）我们直接用一个例子展示它的工作方式。如前所述，序列赋值通常要求左边的目标数量与右边对象中条目的数量完全相等。如果长度不一致就会报错，除

非像上一节那样在右边手动切片：>>> seq = [1, 2, 3, 4]
>>> a, b, c, d = seq >>> a, d (1, 4) >>> a, b = seq
ValueError: too many values to unpack (expected 2) 不过，我们可以在这个例子里用一个星号目标来更灵活地匹配。在下面这次会话的延续中，a 匹配序列的第一个条目，b 匹配其余部分：>>> a, b = seq >>> a 1 >>> b [2, 3, 4]
] 使用星号目标（如这里的名字 b）时，左边的条目数不必与右边可迭代对象（如这里的序列 seq）的长度匹配。事实上，星号目标可以出现在左边的任何位置。例如，在下面的交互中，b 匹配序列的最后一个条目，a 匹配最后一个之前的所有内容：>>> a, b = seq >>> a [1, 2, 3] >>> b 4
当星号目标出现在中间时，它收集列出的其他目标之间的所有内容。因此，在下面的交互中，a 和 c 被赋为第一和最后一个条目，b 得到它们之间的所有内容：>>> a, b, c = seq >>> a 1 >>> b [2, 3] >>> c 4
更一般地，无论星号目标出现在哪里，它都会被赋予一个收集该位置上所有未被赋值条目的列表：>>> a, b, c = seq >>> a 1 >>> b 2 >>> c [3, 4]
自然，和普通序列赋值一样，扩展解包语法也适用于任何序列类型（确切地说，又是任何可迭代对象），而不只是列表。下面是它对字符串中的字符以及可迭代对象 range——按需生成 0...3 的值——进行解包：>>> a, b = 'hack' >>> a, b ('h', ['a', 'c', 'k']) >>> a, b, c = 'hack' >>> a, b, c ('h', ['a', 'c'], 'k') >>> a, b, c = range(4) >>> a, b, c (0, [1, 2], 3)
这与切片在精神上类似，但并非完全相同——序列解包赋值匹配到的条目总是返回列表，而切片返回的是与被切对象类型相同的序列：>>> S = 'hack' >>> a, b, c = S[0], S[1:-1], S[-1] # 切片保留类型 >>> a, b, c ('h', 'ac', 'k') >>> a, b, c = S # 而总是返回列表 >>> a, b, c

('h', ['a','c'],'k') 最后，上一节的收尾例子在这项扩展之下变得更简单了，因为我们不用手动切片就能得到第一条和其余条目（尽管在第一个之后，“其余部分” L 总是变成一个列表）：

```
>>> L = [1, 2, 3, 4] >>> while L: ... front, L = L
# 不切片就拿到头部和其余部分... print(front, L) ... 1 [2, 3, 4] 2 [3, 4] 3 [4] 4 []
```

边界情况（boundary cases）虽然扩展解包很灵活，但一些边界（非典型）情况值得注意。第一，星号目标可能只匹配单个条目，但它总是被赋成列表：

```
>>> seq = [1, 2, 3, 4] >>> a, b, c, d = seq >>> print(a, b, c, d) 1 2 3 [4]
```

第二，如果没有剩余条目匹配星号目标，无论它出现在哪里，都会被赋成一个空列表。下面，名字 a、b、c、d 已经匹配了序列中的每个条目，但 Python 把 e 赋成空列表，而不是当作错误：

```
>>> a, b, c, d, e = seq >>> print(a, b, c, d, e) 1 2 3 4 [] >>> a, b, e, c, d = seq >>> print(a, b, c, d, e) 1 2 3 4 []
```

但是，以下几种情况仍会触发错误：星号目标超过一个、值太少且没有星号（如前所述）、以及星号目标没有被写在一个序列内部：

```
>>> a, b, c, d = seq
SyntaxError: multiple starred expressions in assignment
>>> a, b = seq
ValueError: too many values to unpack (expected 2)
>>> a = seq
SyntaxError: starred assignment target must be in a list or tuple
>>> a, = seq
# 省略括号的单元元素元组
>>> a
# 等价于 a = seq, 但做了一次新拷贝！[1, 2, 3, 4]
```

严格来说，“= 左边星号目标”只能出现一次”的规则，在两边都有嵌套时其实只适用于每一层序列——但你仍然不能在单个嵌套序列中出现超过一个星号：

```
>>> [(a, b), (c, d)] = [(1, 2, 3, 4), (5, 6, 7, 8)] >>> a, b, c, d = (1, [2, 3, 4], 5, [6, 7, 8]) >>> [(a, b, x), (c, d)] = [(1, 2, 3, 4), (5, 6, 7, 8)]
SyntaxError
```

r: multiple starred expressions in assignment 最后，任何赋值目标都可以带星号——虽然你在真实代码中很难找到这么用的：

```
>>> a, L = [1, 2, 3] # 名字目标>>> a, L (1, [2, 3])>>> a, L[0] = [4, 5, 6] # 索引目标>>> a, L (4, [[5, 6], 3])>>> a, L[1:] = [7, 8, 9] # 切片目标>>> a, L (7, [[5, 6], 8, 9])>>> class C: ... # 见第六部分>>> a, C.attr = 'yikes' # 属性目标>>> a, C.attr ('y', ['i','k','e','s'])
```

实用的便利特性 (a useful convenience) 请记住，扩展解包赋值只是一种便利（而且在方便之外，还很长一串）。我们通常可以用显式的索引和切片达到同样效果，但扩展解包写起来更简单。例如常用的“第一个，其余”拆分编码模式，两种写法都行，但切片要多做事：

```
>>> seq [1, 2, 3, 4]>>> a, b = seq # 第一个，其余>>> a, b (1, [2, 3, 4])>>> a, b = seq[0], seq[1:] # 不带的“第一个，其余”>>> a, b (1, [2, 3, 4])
```

同样常用的“其余，最后一个”拆分模式也可以两种写法，但扩展解包语法明显更省按键（也明显更省脑细胞）：

```
>>> a, b = seq # 其余，最后一个>>> a, b ([1, 2, 3], 4)>>> a, b = seq[:-1], seq[-1] # 不带的“其余，最后一个”>>> a, b ([1, 2, 3], 4)
```

由于更简单也更自然，扩展解包语法在 Python 代码中也很常见。应用到 for 循环 (application to for loops) 因为 for 循环语句中的循环变量可以是任何赋值目标，扩展解包在这里也同样适用。我们第 4 章简要用过 for 循环迭代工具，第 13 章会正式学习它，但这里与本章的关联很直接：扩展解包赋值可能出现在 for 关键字之后，而那里更常用的是一个简单变量名：for (a, b, c) in [(1, 2, 3, 4), (5, 6, 7, 8)]: ... 在这种上下文中，每次迭代时 Python 只是把下一个值元组赋给目标元组。例如在第一次循环时，就如同执行了下面这条赋值语句：a, b,

`c = (1, 2, 3, 4) # b` 得到 `[2, 3]` 名字 `a`、`b`、`c` 可以在循环代码中使用，以引用提取出的分量。事实上，这根本不是特例，而只是通用赋值在起作用的一个实例。例如本章前面我们看到，在循环里可以用简单的序列（元组）赋值做类似的事：`for (a, b, c) in [(1, 2, 3), (4, 5, 6)]: ... # a, b, c = (1, 2, 3), ...` 而且我们总能通过手动切片模拟扩展解包赋值的行为：`for all in [(1, 2, 3, 4), (5, 6, 7, 8)]: ... a, b, c = all[0], all[1:-1], all[-1]` 由于我们对 `for` 循环语法的了解还不足以深入细节，这个话题先搁置，等第 13 章再续。

深度理解

·核心概念：`X` 是“可变长度收集器”，收集未被其他目标匹配的所有条目并打包成列表。位置不限（开头/中间/结尾），目标永远是列表——哪怕只收集到一条，甚至零条（空列表）。

·底层实现：`a, b = seq` 编译为按长度的解包字节码 `UNPACKEX`（Python 3.11 之后；此前为 `UNPACKSEQUENCE + BUILDLIST` 组合）：解释器先算出固定目标需要的个数 `k`，把 `b` 的位置与长度差值 `len(seq) - k` 结合，把中间那一段整体打包成列表。这也是为什么 `b` 永远得到列表——构造列表是字节码里写死的动作。

·设计原因：PEP 3132（2006，Python 3.0）引入。它替代了“切三个片、拼两个串”的笨拙写法，把“取头&取尾&取中间”这种序列数据结构最常见的操作语法化。

·实际问题：`first, rest = records` 处理 CSV 表头；`body, last = sentence` 取出最后一个词；`for k, v in data` 在循环里做变长字段解包；`a, , b = row` 跳过中间字段（用当

垃圾桶)。这是 Python 程序员每天都在用的语法。

·初学者误区：①以为 `b` 得到的是元组或原类型——其实永远是列表；②想用两个例如 `a, b, c = ...`——语法错误，一次只能一个；③`a = seq` 也是语法错误，必须像 `a, = seq` 这样待在一个序列里（靠逗号构成单元素元组），但注意它会复制一份列表到 `a`。

·边界细节：`a, = seq` 看似平凡，却因为 `a` 绑定到新列表而生成一份拷贝；嵌套结构里每一层各可有一个，例如 `[(a, b), (c, d)] = ...` 合法，而同一层里两个非法。

代码分析

```
>>> seq = [1, 2, 3, 4]
>>> a, *b = seq          # + a = 1, b = [2, 3, 4]
>>> *a, b = seq          # + a = [1, 2, 3], b = 4
>>> a, *b, c = seq       # a = 1, b = [2, 3], c = 4
>>> a, b, c, *d = seq    # b [4]
>>> a, b, c, d, *e = seq  # e []
>>> a, *b, c, *d = seq    # *
SyntaxError: multiple starred expressions in assignment
>>> *a, = seq            # a seq
```

解释器如何处理：对 `a, b, c = seq`，解释器先把“固定位置”的数量算出来：`a` 与 `c` 各占 1，需要至少 2 个元素；然后调用序列迭代器，把第 1 项给 `a`、最后 1 项给 `c`，中间 `2..len-2` 的全部收集到 `b` 中并构造成列表对象。整个过程假设长度够用：若 `seq` 长度小于 2 会报 `ValueError: not enough values to unpack`。`a, L[0] = [4,5,6]` 这种是“目标本身也带下标”的奇特组合：先综合解包逻辑算出 `a = 4`，`[5,6] = L[0]`，再对列表 `L` 执行 `L.setitem(0, [5,6])`——

—可见语法与普通赋值目标体系完全正交，可以自由组合

。

11.7 旁注：Python 中众多的星号 (THE MANY STARS OF PYTHON)

英文原文

No, this sidebar is not about personalities (which already permeate software culture more than they should). It's a reference to all the special-case appearances of that have cropped up in Python over the years, many of which you've already seen. Prior to Python 3.5, the special starred-item syntax forms can appear in: Assignments, where a single `X` starred assignment target of any kind (name, index, slice, attribute), and repeatable in nested parts, collects unmatched items in a new list, as covered in this chapter. Function headers, where single `X` and `X` starred names collect unmatched positional and keyword arguments in a tuple and dictionary, respectively. Function calls, where single `X` and `X` starred expressions unpack iterables and mappings into individual positional and keyword arguments, respectively. As of Python 3.5, function calls (the latter listed item) support any number of `X` and `X` unpacking expressions, and any number of `X` and `X` starred expressions can also appear in object literals, where they unpack collections of types implied by the literal

in which they appear: `[x, iter]` # List: unpack items in iterables `(x, iter, y)` # Tuple: ditto, parentheses or not `{iter, x}` # Set: ditto, though values are unordered and unique `{x: y, map}` # Dict: unpack keys/values in mappings, rightmost dup wins As of Python 3.10, both `X` and `X` starred names can also show up in the patterns of the match statement covered in the next chapter, where they serve roles similar to that in sequence assignment, though only `X` overlaps between the two, and its assignment in match is really a side effect of a true-or-false test. And as of Python 3.11, an `except` can appear in try statements, where it allows multiple handlers to be run to process multiple exceptions wrapped in an exception group—an addition that bifurcates the already-convoluted try for narrow roles, which we won't meet until Chapter 35. All that being said, the avenue of stars has been a meandering walk in Python, and it's not impossible that other parts of the language will rise to stardom in the future. Consult the stars for more info.

中文翻译

不，这个旁注可不是在讲名人（名人已经渗透软件文化过头了）。它说的是这些年来 Python 中冒出来的、的各种特例用法，其中很多你已经见过了。在 Python 3.5 之前，特殊的星号条目语法形式可以出现在：赋值（assignments）——单个 `X` 星号赋值目标可以是任意种类（名字、索引、切片、属性），并且可以在嵌套部分中重复出现，它把未被匹

配的条目收集进一个新列表，正如本章所讲。函数头 (function headers) ——单个 X 和 X 星号名字分别把未匹配的位置参数和关键字参数收集进一个元组和一个字典。函数调用 (function calls) ——单个 X 和 X 星号表达式分别把可迭代对象和映射解包成各个独立的位置参数和关键字参数。从 Python 3.5 起，函数调用 (上面列出的最后一项) 支持任意数量的 X 和 X 解包表达式，而且任意数量的 X 和 X 星号表达式也可以出现在对象字面量 (object literal) 中，在那里它们解包与所在字面量类型对应的集合：`[x, iter]` # 列表：解包可迭代对象中的条目`(x, iter, y)` # 元组：同样，有无括号均可`{iter, x}` # 集合：同样，不过值是无序且唯一的`{x: y, map}` # 字典：解包映射中的键/值，最右边的重复键胜出从 Python 3.10 起，X 和 X 星号名字也可以出现在下一章将讲的 `match` 语句的模式中，在那里它们的作用与序列赋值中的类似，不过两者之间只有 X 有重合，而且它在 `match` 中的赋值其实是真假测试的副作用。从 Python 3.11 起，`except` 可以出现在 `try` 语句中，它允许运行多个处理器来处理封装在异常组 (exception group) 中的多个异常——这个新增使本来就很绕的 `try` 在狭窄用途上又分叉了，我们要到第 35 章才会碰到它。总而言之，Python 的星号之路是一场蜿蜒的旅程，未来语言的其他部分也不是不可能"晋升为星"。想了解更多，就仰望星空吧。

深度理解

·核心概念：是 Python 的"万能展开符"，在多处语法点有不同的语义：赋值里收集成列表、函数调用里展开成实参、函数定义里收拢成形参、字面量里展开成元素、`match` 模式里做通配匹配。

·底层实现：函数调用 `f(args)` 编译为 `CALLFUNCTIONEX`（展开元组参数表）；`f(kwargs)` 展开为关键字映射；字面量展开则是专门的处理指令（如 `BUILDLIST` 前先展开迭代器）。`map` 在字典字面量里对应 `DICTUPDATE/DICTMERGE` 字节码，最右重复键胜出由此而来。

·设计原因：同一个符号复用是语言演进的历史沉积：最早只有函数调用的 `args/kwargs`，后来 PEP 3132（赋值）、PEP 448（3.5 全面展开）、PEP 634 `match` 逐步扩展。它们共享“一个符号表示集合”的直觉，代价是需要逐个记忆语境。

·实际问题：写 `print(items)` 一次输出整个列表、`dict(a, b)` 合并字典、`def f(a, kw)` 收拢可选参数——这些都在生产代码中随处可见。

·初学者误区：认为只用于函数。其实“星号只有一种”是完全错误的印象——它有收集（收）和展开（放）两个方向的语义，还有赋值、调用、定义三种上下文。本节把全部登场场合提前罗列，正是为了让你建立一个“星号全景图”。

11.8 多重目标赋值 (Multiple-Target Assignments)

英文原文

Simpler than sequence assignment on first glance, a multiple-target assignment simply assigns all the giv

en targets to the same object all the way to the right. The following, for example, assigns the three names (variables) a, b, and c to the string 'code':

```
>>> a = b = c = 'code'>>> a, b, c ('code','code','code')
```

This form is equivalent to—but easier to code and lighter on line count than—these three assignments:

```
>>> c = 'code'>>> b = c>>> a = b
```

Multiple-target assignment and shared references While this seems simple, keep in mind that there is just one object here, shared by all three variables: they all wind up referencing the same object in memory. This behavior is worry-free for immutable types—for example, when initializing a set of counters to zero (recall that variables must be assigned before they can be used in Python, so you must initialize counters to zero before you can start adding to them):

```
>>> a = b = 0>>> b = b + 1>>> a, b (0, 1)
```

Here, changing b changes only b because numbers do not support in-place changes. As long as the object assigned is immutable, it's irrelevant if more than one name references it. As usual, though, we have to be more cautious when initializing variables to an empty mutable object such as a list or dictionary:

```
>>> a = b = []>>> b.append(42)>>> a, b ([42], [42])
```

This time, because `a` and `b` reference the same object, appending to it in place through `b` will impact what we see through `a` as well. This is really just another example of the shared reference phenomenon we first met in Chapter 6. To avoid the issue, initialize mutable objects in separate statements instead, so that each creates a distinct empty object, by running a distinct literal expression:

```
>>> a = [] >>> b = [] >>> b.append(42) # a and b
do not share the same object >>> a, b ([], [42])
```

A tuple assignment like the following has the same effect with more brevity—by running two list-literal expressions, it creates two distinct objects:

```
>>> a, b = [], [] # a and b do not share the same object
```

中文翻译

多重目标赋值乍看比序列赋值更简单：它只是把所有给出的目标赋给最右边唯一的那个对象。例如下面把三个名字（变量）`a`、`b`、`c`都赋给字符串`'code'`：`>>> a = b = c = 'code'``>>> a, b, c ('code','code','code')` 这种形式等价于——但比——这三条赋值更容易编写、行数也更少：`>>> c = 'code'``>>> b = c``>>> a = b` 多重目标赋值与共享引用（multiple-target assignment and shared references）虽然这看起来简单，但请记住：这里只有一个对象，被三个变量共享——它们最终都引用内存中的同一个对象。对于不可变（immutable）类型，这种行为可以放心——例

如把一组计数器初始化为零（回顾一下：在 Python 中变量必须赋值后才能使用，所以你必须先把计数器初始化为零才能开始往上加）：
`>>> a = b = 0`
`>>> b = b + 1`
`>>>`
`a, b` (0, 1) 这里，改变 `b` 只改变 `b`，因为数字不支持原地修改。只要被赋的对象是不可变的，多个名字引用它就没关系。但一如既往，当把变量初始化为一个空的可变（mutable）对象（如列表或字典）时，我们就得更加小心了：
`>>>`
`a = b = []`
`>>> b.append(42)`
`>>> a, b` ([42], [42]) 这一次，因为 `a` 和 `b` 引用同一个对象，通过 `b` 对它做原地追加，也会影响我们从 `a` 看到的内容。这其实只是第 6 章首次遇到的共享引用（shared reference）现象的又一个例子。要避免这个问题，请用互不相干的语句分别初始化可变对象，这样每条语句通过运行各自的字面量表达式创建一个独立的空对象：
`>>> a = []`
`>>> b = []`
`>>> b.append(42)`
`a` 和 `b` 不共享同一个对象
`>>> a, b` ([], [42]) 像下面这样的元组赋值有同样的效果且更简洁——通过运行两个列表字面量表达式，它创建了两个独立的对象：
`>>> a, b = [], []`
`a` 和 `b` 不共享同一个对象

深度理解

·核心概念：`a = b = c = value` 把同一个对象的引用赋给所有名字（从右到左依次绑定）。一句话总结：无量词，只有一个对象，大家共享它。

·底层实现：字节码先计算最右边的值，压栈后用 DUPTOP（复制栈顶）配合多次 STORE 依次存到 `c`、`b`、`a`——全程只有一份对象，没有拷贝。这解释了为什么顺序是“先 `c` 后 `a`”（等价于 `c='code'; b=c; a=b`）。

·设计原因：初始化多个计数器/标志位时一行写完。但它故意保留"共享引用"语义，而非像某些语言那样复制——因为 Python 一切赋值皆引用，别无选择。

·实际问题：width = height = margin = 0 初始化配置；d = c = b = a = {} 若用于搭建嵌套默认值则是典型"陷阱现场"（会全部指向同一字典）。

·初学者误区：①认为 a = b = [] 创建了两个独立列表——只有一个空列表，a 与 b 是它的两个名字；②以为 b = b + 1 会改变 a——不会，因为数字不可变，b 只是重新绑定到新整数，a 原地不动；③习惯性写 a, b = [], [] 来刻意制造两个独立对象，却不知道为什么要这么做。

代码分析

```
>>> a = b = c = 'code'          # cba
>>> a = b = 0                    #
>>> b = b + 1                    # b 1a 0
>>> a, b
(0, 1)

>>> a = b = []                  #
>>> b.append(42)                 # a "" 42
>>> a, b
# ([42], [42])

>>> a = []                      #
>>> b = []
>>> a, b = [], []               # ->
```

解释器如何处理：a = b = c = 'code' 中'code' 字面量常量（intern 驻留）成一个字符串对象，随后 STORENAME c、STORENAME b、STORENAME a 三条指令把同一对象引用写入三个名字槽。对 a = b = [], [] 每次执行

确实新建一个列表——但只新建一次，因为这里只有一条 STORE 的目标表达式。而 `a, b = [], []` 里有两个 `[]` 字面量，各自求值，所以生成两个对象。这是理解“哪些字面量被求值几次”的关键：求值发生在赋值运算符的右侧，一个右侧表达式对应一份新建对象。

11.9 增强赋值 (Augmented Assignments)

英文原文

In addition to the basic, sequence (original and starred), and multiple assignment forms already covered, Python gained all the assignment statement formats listed in Table 11-2 relatively early in its career. Known as augmented assignments, and borrowed from the C language, these formats are mostly just shorthand: they imply the combination of a binary (two-operand) expression and an assignment.

For instance, the following two formats are functionally equivalent, though the latter may use in-place options to change X directly, as you'll see in a moment:

`X = X + Y` # Basic form `X += Y` # Augmented form

Table 11-2. Augmented assignment statements

`X += Y` `X -= Y` `X = Y` `X /= Y` `X @= Y` `X //= Y` `X %= Y` `X = Y`
`X >>= Y` `X <<= Y` `X &= Y` `X |= Y` `X ^= Y`

Augmented assignment works on any type that supports

orts the implied binary expression. For example, here are two ways to add 1 to a name; really, they both change a name to reference different values:

```
>> x = 1 >> x = x + 1 # Basic >> x 2 >> x += 1 # Augmented >> x 3
```

When applied to a sequence such as a string, the augmented form performs concatenation instead, because that's what + means for such objects. Thus, the second line here is equivalent to typing the longer `S = S + 'HACK'`:

```
>> S = 'hack' >> S += 'HACK' # Implied concatenation >> S 'hackHACK'
```

As shown in Table 11-2, there are analogous augmented assignment forms for other Python binary expression operators (i.e., operators with values on their left and right sides). For instance, `X = Y` multiplies and assigns, `X >>= Y` shifts right and assigns, and so on (though per Chapter 5, the form `X @= Y` is unused and unimplemented by Python itself today).

All told, augmented assignments have three noteworthy advantages: 1. There's less for you to type. (Need this book say more?) 2. The left side has to be evaluated only once. In `X += Y`, `X` may be a complicated object expression. In the augmented form, its code must be run only once. However, in the long form, `X = X + Y`, `X` appears twice and must be run twice. Because of this, augmented assignments usually run faster. 3.

The optimal technique is automatically chosen. That is, for objects that support in-place changes, the augmented forms automatically perform in-place change operations instead of slower copies.

The last point here requires a bit more explanation. For augmented assignments, in-place operations may be applied for mutable objects as an optimization. Recall that lists can be extended in a variety of ways. To add a single item to the end of a list, we can concatenate or call `append`:

```
>> L = [1, 2] >> L = L + [3] # Concatenate one: slower
>> L [1, 2, 3] >> L.append(4) # Faster, but in place
>> L [1, 2, 3, 4]
```

And to add a set of items to the end, we can either concatenate again or call the `list extend` method:

```
>> L = L + [5, 6] # Concatenate many: slower
>> L [1, 2, 3, 4, 5, 6] >> L.extend([7, 8]) # Faster, but in place
>> L [1, 2, 3, 4, 5, 6, 7, 8]
```

In both cases, concatenation is less prone to the side effects of shared object references but will generally run slower than the in-place equivalent. Concatenation operations must create a new object, copy in the list on the left, and then copy in the list on the right. By contrast, in-place method calls simply add items at the end of a memory block (it can be a bit more complicated than that internally, but this description suffices).

When we use augmented assignment to extend a list, we can largely forget these details—Python automatically calls the quicker `extend` method (or its equivalent) instead of using the slower concatenation operation implied by `+`:

```
>> L += [9, 10] # Mapped to L.extend([9, 10]) >> L [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

As suggested in Chapter 6, we can also use slice assignment (e.g., `L[len(L):] = [11,12,13]`), but this works roughly the same as the simpler and more mnemonic list `extend` method or the `+=` statement. Note, however, that because of this equivalence `+=` for a list is not exactly the same as `a +` and `=` in all cases—for lists `+=` allows arbitrary sequences (just like `extend`), but concatenation normally does not:

```
>> L = [] >> L += 'hack' # += and extend allow any sequence, but + does not! >> L ['h','a','c','k'] >> L = L + 'code'
TypeError: can only concatenate list (not "str") to list
```

Moreover, using index and slice targets in augmented assignment may change mutables in other ways:

```
>> L = [1, 2] >> L[0] += 10 # Change an item of a list, in place
>> L [11, 2] >> L[-1:] += [3, 4] # Change a section of a list, in place
>> L [11, 2, 3, 4]
```

Augmented assignment and shared references More subtly, the in-place change implicit in `+=` for lists is v

ery different from the new object created by + concatenation. As for all shared-reference cases, this difference may matter when objects are shared by many names:

```
>> L = [1, 2] >> M = L # L and M reference the same object
>> L = L + [3, 4] # Concatenation makes a new object
>> L, M # Changes L but not M ([1, 2, 3, 4], [1, 2])
>> L = [1, 2] >> M = L >> L += [3, 4] # But += really means extend, not +
>> L, M # M sees the in-place change too! ([1, 2, 3, 4], [1, 2, 3, 4])
```

This only matters for mutables like lists and dictionaries, and it is a fairly obscure case (at least, until it impacts your code!). As always, make copies of your mutable objects if you need to break the shared reference structure.

中文翻译

除了已经讲过的基本、序列（原始的和带星号的）和多重赋值形式之外，Python 在其早期就引入了表 11-2 中列出的所有赋值语句格式。它们被称为增强赋值（augmented assignment），借自 C 语言，这些格式大多只是简写：它们暗示了一个二元（两个操作数）表达式与一次赋值的组合。例如，下面两种写法功能等价，不过后者可能使用原地操作直接修改 X，你稍后就会看到：X = X + Y # 基本形式 X += Y # 增强形式表 11-2：增强赋值语句（对应 Python 每种二元运算符：加法、减法、乘法、除法、矩阵乘法 @=、整除、取模、幂、右移、左移、按位与、按位或、按位异或）增强赋值适用于任何支持对应二元表达式的类型。例如，

下面两种方法都给一个名字加 1；其实，它们都只是让名字改引用到不同的值：

```
>>> x = 1
>>> x = x + 1 # 基本形式
>>> x 2
>>> x += 1 # 增强形式
>>> x 3
```

当作用于字符串这类序列时，增强形式做的是拼接（concatenation），因为对这类对象 + 的含义就是拼接。因此，这里的第二行等价于写更长的 `S = S + 'HACK'`：

```
>>> S = 'hack'
>>> S += 'HACK' # 隐含的拼接
>>> S 'hackHACK'
```

如表 11-2 所示，Python 其他二元表达式运算符（即左右两侧都有值的运算符）也有类似的增强赋值形式。例如 `X = Y` 是乘后赋，`X >>= Y` 是右移后赋，依此类推（不过按第 5 章的说明，`X @= Y` 今天还是被 Python 闲置且未实现的形式）。总而言之，增强赋值有三个值得注意的优点：

1. 输入更少。（还需要这本书多说吗？）
2. 左侧只求值一次。在 `X += Y` 中，`X` 可能是一个复杂的对象表达式。在增强形式中，它的代码只需运行一次。然而在长形式 `X = X + Y` 中，`X` 出现了两次，必须运行两次。正因如此，增强赋值通常跑得更快。
3. 自动选择最优技术。也就是说，对于支持原地修改的对象，增强形式会自动执行原地修改操作，而不是较慢的拷贝。最后一点需要更多解释。对于增强赋值，原地操作可能会被应用于可变对象作为一种优化。回忆一下，列表有多种扩展方式。要给列表末尾加一项，我们可以拼接或调用 `append`：

```
>>> L = [1, 2]
>>> L = L + [3] # 拼接一条：较慢
>>> L [1, 2, 3]
>>> L.append(4) # 较快，但是原地
>>> L [1, 2, 3, 4]
```

要给末尾加一组条目，我们可以再次拼接或调用列表的 `extend` 方法：

```
>>> L = [1, 2, 3, 4, 5, 6] # 拼接多条：较慢
>>> L [1, 2, 3, 4, 5, 6]
>>> L.extend([7, 8]) # 较快，但是原地
>>> L [1, 2, 3, 4, 5, 6, 7, 8]
```

两种情况下，拼接都不太容易受到共享对象引用副作用的影响，但通

常比原地等价写法慢。拼接操作必须创建一个新对象，把左边的列表拷进去，再把右边的列表拷进去。相比之下，原地方法调用只是在一个内存块的末尾追加条目（内部可能比这复杂一点，但这样描述足够了）。当我们用增强赋值扩展列表时，大体可以忘掉这些细节——Python 会自动调用更快的 `extend` 方法（或其等价机制），而不是用 `+` 隐含的较慢拼接操作：

```
>>> L += [9, 10] # 映射为 L.extend([9, 10])
>>> L [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

正如第 6 章提到的，我们还可以用切片赋值（例如 `L[len(L):] = [11,12,13]`），但它与更简单、更好记的列表 `extend` 方法或 `+=` 语句的效果大体相同。不过请注意，正因为这种等价，对列表来说 `+=` 并不在所有情况下都等同于 `+` 再加 `=`——对列表，`+=` 允许任意的序列（正如 `extend`），但拼接通常不允许：

```
>>> L = []
>>> L += 'hack' # += 和 extend 允许任意序列，但 + 不允许！
>>> L ['h','a','c','k']
>>> L = L + 'code'
TypeError: can only concatenate list (not "str") to list
```

此外，在增强赋值中使用索引和切片目标可能以其他方式修改可变对象：

```
>>> L = [1, 2]
>>> L[0] += 10 # 原地修改列表的一项
>>> L [11, 2]
>>> L[-1:] += [3, 4] # 原地修改列表的一段
>>> L [11, 2, 3, 4]
```

增强赋值与共享引用（augmented assignment and shared references）更微妙的是，`+=` 对列表隐含的原地修改，与 `+` 拼接创建的新对象非常不同。和所有共享引用情况一样，当对象被多个名字共享时，这种差异就变得重要了：

```
>>> L = [1, 2]
>>> M = L # L 和 M 引用同一个对象
>>> L = L + [3, 4] # 拼接创建了一个新对象
>>> L, M # 改变了 L 但没有改变 M
([1, 2, 3, 4], [1, 2])
>>> L = [1, 2]
>>> M = L
>>> L += [3, 4] # 但 += 的真正含义是 extend，而不是 +
>>>
```

L, M # M 也看到了原地修改! ([1, 2, 3, 4], [1, 2, 3, 4]) 这只对可变对象 (如列表和字典) 有影响, 而且是一个相当隐蔽的案例 (至少, 在它影响你的代码之前!)。一如既往, 如果需要打破共享引用结构, 请复制你的可变对象。

深度理解

·核心概念: $X += Y$ 不是 $X = X + Y$ 的简单缩写, 而是一个有语义差异的运算符: 对可变对象优先走"原地修改" (in-place) 路径。这才是本章最"反直觉"也最重要的一个点。

·底层实现: $a += b$ 会先尝试 $a.iadd(b)$ (原地加, 字节码 INPLACE); 只有未定义 $iadd$ 时才回退到 $a.add(b)$, 把结果重新绑定给 a 。对列表: $+=$ 实际调用 $extend$ (等价 $iadd$), 所以允许任意可迭代对象而 $+$ 不行; 对整型/字符串: 没有 $iadd$, $+=$ 落实为 $a = a.add(b)$, 即复制语义。 $L[0] += 10$ 的字节码是 `LOAD L; LOADCONST 0; DUP TWO; LOADATTR...再 STORESUBSCR`——"先取再算再存回", 三者如一体的一条指令。

·设计原因: 参考 C 语言加入 $+=$, 本意是省字; Python 更进一步, 把"原地修改"与"复制"的抉择交给对象的魔法方法, 让解释器替程序员自动选出更快的路径——用户不用操心何时用 `append` 何时用 $+$ 。

·实际问题: 性能 (大量字符串拼接用 `join`, 大量列表累积用 $+=/extend$); $L[:0] += ...$ 头部插入; 但最致命的应用是共享引用: $L += [3,4]$ 会传染所有共享者, $L = L + [3,4]$ 则安全。

·初学者误区：①以为 += 恒等于 + 和 = 的组合；②L += 'hack' 能成功而 L = L + 'hack' 报 TypeError——原因正是 += 走 extend、+ 走拼接，序列兼容规则不同；③对不可变对象（数字、字符串）认为 += 会“原地修改”——不会，它们重新绑定新对象；④L.sort()/L.append() 返回 None，绝不要 L = L.append(x)。

代码分析

```
>>> x = 1
>>> x += 1          # __iadd__ x = x.__add__(1) 2
>>> S = 'hack'
>>> S += 'HACK'     # __iadd__ 'hackHACK'

>>> L = [1, 2]
>>> L += [9, 10]    # __iadd__ L.extend([9, 10])
>>> L += 'hack'     # extend ['h','a','c','k']
>>> L = L + 'code'  # list
TypeError: can only concatenate list (not "str") to list

>>> L[0] += 10      # L[0] 10 L[0]—
>>> L[-1:] += [3, 4] #

>>> L = [1, 2]; M = L
>>> L += [3, 4]     # extend M ""
>>> L, M           # ([1, 2, 3, 4], [1, 2, 3, 4])
```

解释器如何处理：L[0] += 10 的伪字节码：LOADFAST L; LOADCONST 0; DUPTOPTWO; BINARYOP +; STORES UBSR。DUPTOPTWO 把“容器 L”和“下标 0”都复制一份，确保计算 L[0]+10 和写回 L[0]=... 用的是同一个容器对象和同一个下标、且在单条指令内完成——这其实和 C 语言 ++ 的“读-改-写不可分割”精神一致（Python 里叫 a

ugmented 原子的容器更新)。当目标是一个带副作用的复杂表达式 (如 `d['k'].items[0] += 1`) 时, `+=` 只求值目标一次的优势更明显——长形式要求值两次, 可能在不同时刻得到不同对象, 导致诡异 bug。

11.10 命名赋值表达式 (Named Assignment Expressions)

英文原文

For most of Python's three-decade tenure, it resisted emulating the assignment-as-expression idiom of the C language, on the grounds that it was too subtle and error-prone, and fostered code that was hard to read. While these concerns still apply, Python 3.8 gained a flavor of this, known as named assignment. Importantly, this flavor does not make normal `=` assignment statements nestable expressions, as they are in C.

Instead, it adds a new expression operator to Python, `:=`, on the grounds that its different and limited syntax may neutralize pitfalls of other languages. You still can't accidentally type `=` when you mean `==`, and `:=` is visually distinct.

We explored this expression briefly in a spoiler note in the previous chapter.

In short, the expression `name:= value` first evaluates expression value, and then both: 1. Assigns the result

to the provided variable name 2. Returns the result as the value of the overall:= expression There's no reason to use this syntax for the first part alone, because all of Python's = assignment statements already assign values to names (in fact, you'll get a syntax error if you try using:= as a statement sans parentheses).

Because:= is an expression that returns the value assigned, though, it can be nested in contexts where statements aren't allowed and may be used in roles that both test and then use the assigned name.

As an artificial first example, the following nested:= both assigns 2 to b, and returns it to be used in the string-repetition result assigned to a:

```
>> a='hack' (b:= 2) >> a'hackhack'>> b 2
```

Among its deliberate limitations, named assignment allows only a simple, single name (variable) on the left, so neither other assignment-statement forms nor references to data-structure components work here:

```
>> (python:= 3.12) + 0.01 # Just a name on the left
3.13 >> python 3.12 >> (python, Python:= 3.12, 3.13)
+ 0.01 TypeError: can only concatenate tuple (not "float") to tuple
>> (python[0]:= 3.12) + 0.01 SyntaxError: cannot use assignment expressions with subscript
>> (python.attr:= 3.12) + 0.01 SyntaxError: cannot use assignment expressions with attribute
```

The first of these failers, for example, is taken to be

a three-item tuple with `:=` in the middle, not a two-tuple named sequence assignment. Moreover, named assignment does not support any of the augmented assignment forms we met earlier: there is no `+=`, for instance, and coding statement `X += 1` may actually take less work than expression `(X := X + 1)` (subtly, `a := +` runs, but simply applies the `+` identity operator to the expression on the right).

When to use named assignment While more useful contexts for `:=` are limited, its most common use cases arise in concert with the `if` and `while` statements, which were both introduced in the preceding chapter and earlier. Without `:=`, it was—and still is—common to assign a variable before the statement, test it in the statement header, and then use it in the statement body

For example, code that reads lines from files and must detect the end of the file (which, you'll recall from Chapter 9, means an empty string that is logically false) may look like these partial snippets (to run code in this section live, open a text file for input and assign it to `file` before each snippet):

```
line = file.readline() # Sans the := expression if line:
print(line) line = file.readline() # Ditto, in while loops
while line: print(line) line = file.readline() To avoid redundant calls, it's also common to code the latter like this in Python—as in the preceding chapter:
```

while True: # Sans both := and redundancy line = file.readline() if not line: break print(line) These forms still work well, and the := is never required. With :=, however, we can sometimes collapse a fetch, assignment, and test into a one-liner within statement headers themselves:

if line := file.readline(): # The := alternatives print(line) while line := file.readline(): # Read all lines (vs: for line in file) print(line) In both cases, because := returns the results of the readline call, its logical value can be tested in the statement header itself. And because := also assigns the result to line, it can be used in the statement's body if it is run. The upshot is that := provides a sort of shortcut that allows multiple operations to be coded in a compact way.

If you opt to use this expression, bear in mind that it often requires enclosing parentheses to avoid interacting with surrounding code. Even comparing its result to an explicit value, for instance, mandates parentheses that are not required in the equivalent statements:

if (line := file.readline()) != ignore: # Parentheses required print(line) while (line := file.readline()) != stop: # And not a bad idea print(line) As a rule of thumb, if you opt to use :=, wrapping it in parentheses both sets it off visually and avoids sticky issues that may arise if the code surrounding it changes its meaning unexp

ectedly. In the preceding, for example, names are assigned the results of the `!=` tests if parentheses are omitted, because it binds tighter than `:=` (see Chapter 9). Moreover, `:=` has special syntax rules that we'll omit here, but they encourage parenthesized usage by design.

Besides file reads and similar roles, the `:=` can be leveraged to reuse results in literals and get last results in comprehensions and other iteration tools, and can even be nested in f-strings and itself—with requisite parentheses:

```
>> [val:='Py!', val 2, val 3] # Reusing a value ['Py!', 'Py!Py!', 'Py!Py!Py!']
```

```
>> list(pow:= 2 num for num in [2, 4, 8]) # Capturing a result [4, 16, 256]
>> pow 256
>> f'Hello {(name:= input('Who are you?'))}' # Nesting in f-strings (hmm)
Who are you? Pat
Hello Pat
>> name'Pat'
>> (x:= (y:= (z:= 1) + 1) + 1) # Nesting in itself (hmm)
2
3
>> x, y, z
(3, 2, 1)
```

As another rule of thumb, bear in mind that a deeply nested and obscure `:=` expression might seem clever but will be a lot harder to read—and even notice—than a simple standalone assignment. You'll be able to judge this for yourself in examples in Chapter 20. Generally speaking, though, for the sake of others (including your future self!), use `:=`, like so many nestable tools, sparingly and wisely. Clarity is usually worth the

extra line or two.

中文翻译

在 Python 三十年的大部分岁月里，它一直拒绝模仿 C 语言的"赋值即表达式"习语，理由是它太微妙、容易出错，并助长了难以阅读的代码。虽然这些顾虑至今仍然成立，但 Python 3.8 还是获得了一种形式，称为命名赋值（named assignment）。重要的是，这种形式不会像 C 语言那样让普通的 = 赋值语句成为可嵌套的表达式。相反，它给 Python 增加了一个新的表达式运算符 :=，理由是它不同而受限的语法或许能化解其他语言的陷阱。你仍然不会在本想写 == 时不小心敲出 =，而且 := 在视觉上明显不同。我们在上一章的一个剧透注记里简要探索过这个表达式。简言之，表达式 name := value 首先求值表达式 value，然后做两件事：1. 把结果赋值给给出的变量名 name 2. 把结果作为整个 := 表达式的值返回单独为了第一部分，没有理由使用这个语法，因为 Python 的所有 = 赋值语句本来就会把值赋给名字（事实上，如果尝试把 := 不带括号当语句用，你会得到语法错误）。但由于 := 是一个返回被赋值值的表达式，它可以嵌套在语句不允许出现的上下文里，也可以在"先测试、后用值"的角色中派上用场。作为第一个（人造的）例子，下面嵌套的 := 既把 2 赋给 b，又把它返回给字符串重复操作的结果并赋给 a：

```
>>> a = 'hack' (b := 2) >>> a'hackhack'>>> b 2
```

在刻意设计的限制中，命名赋值只允许左侧是简单的单个名字（变量），所以其他赋值语句形式或对数据结构组件的引用在这里都不行：

```
>>> (python := 3.12) + 0.01 # 左边只能是名字3.13 >>> python 3.12 >>> (python, Python := 3.12, 3.13) + 0.01
```

TypeError: c

an only concatenate tuple (not "float") to tuple >>> (python[0] := 3.12) + 0.01 SyntaxError: cannot use assignment expressions with subscript >>> (python.attr := 3.12) + 0.01 SyntaxError: cannot use assignment expressions with attribute 例如，上面第一个失败例会被当作一个三项元组、:= 在中间，而不是双目标的命名序列赋值。此外，命名赋值不支持我们前面遇到的任何增强赋值形式：例如没有 +=，而且写语句 X += 1 可能比表达式 (X := X + 1) 更省事（微妙的是，:=+ 能运行，但它只是对右边表达式应用 + 恒等运算符）。何时使用命名赋值（when to use named assignment）虽然 := 更有用的场景有限，但它最常见的用例出现在与 if 和 while 语句配合时，这两者在前一章和更早的地方都介绍过。没有 := 时，过去和现在都常见的写法是：在语句之前给变量赋值，在语句头中测试它，然后在语句体中用它。例如，从文件读取行并必须检测文件结束（还记得第 9 章讲的：这意味着空字符串在逻辑上为假）的代码可能长这样（要现场运行本节代码，请先以输入模式打开一个文本文件并赋给 file，再运行每个片段）：

```

line = file.readline() # 不用 := 表达式
if line: print(line)
line = file.readline() # while 循环中同样
while line: print(line)
line = file.readline()

```

为避免重复调用，在 Python 中把后者写成这样也很常见——就像前一章那样：

```

while True: # 既不用 := 也没有冗余
    line = file.readline()
    if not line: break
    print(line)

```

这些形式仍然很好用，:= 从来不是必需的。不过，有了 :=，我们有时可以把“取数、赋值、测试”压缩成语句头里的一行：

```

if line := file.readline(): # := 的替代写法
    print(line)
while line := file.readline(): # 读所有行
    print(line)

```

（对比：for line in file) print(line) 两种情况

里，因为 `:=` 返回了 `readline` 调用的结果，所以它的逻辑值可以直接在语句头里测试。又因为 `:=` 同时把结果赋值给了 `line`，所以一旦执行到语句体就可以使用它。结论是，`:=` 提供了一种捷径，允许把多个操作紧凑地写在一起。如果你决定使用这个表达式，请记住它常常需要加上括号以避免与周围代码相互作用。例如，即使只是把它与一个显式值比较，也强制要求括号，而等价的语句并不需要：`if (line := file.readline()) != ignore: # 必须加括号 print(line)` 而 `while (line := file.readline()) != stop: # 加上括号也没坏处 print(line)` 作为经验法则，如果选择使用 `:=`，把它包在括号里既能在视觉上把它分隔开，又能避免周围代码意外改变含义时出现的麻烦问题。例如在前面的代码里，如果省略括号，`!=` 比 `:=` 绑定得更紧（见第 9 章），名字会被赋为 `!=` 测试的结果。此外，`:=` 有一些特殊语法规则，我们在这里略过，但它们从设计上就鼓励使用括号。除了文件读取和类似角色，`:=` 还可以用于在字面量中复用结果、在推导式（comprehension）和其他迭代工具中获得最后一个结果，甚至可以在 f-string 中以及它自身内部嵌套——只要带必要的括号：

```
>>> [val := 'Py!', val 2, val 3] # 复用同一个值['Py!', 'Py!Py!', 'Py!Py!Py!']
>>> list(pow := 2 for num in [2, 4, 8]) # 捕获一个结果[4, 16, 256]
>>> pow
256
>>> f'Hello {(name := input('Who are you?'))}' # 嵌套在 f-string 里 (啧啧) Who are you? Pat'Hello Pat'
>>> name
'Pat'
>>> (x := (y := (z := 1) + 1) + 1) # 嵌套在它自身里 (啧啧 2) 3
>>> x, y, z
(3, 2, 1)
```

另一条经验法则：请记住，一个深度嵌套、含义隐晦的 `:=` 表达式也许看起来很聪明，但读起来——甚至注意到它——都比一个简单的独立赋值难得多。第 20 章的例子你可以自行体会。但总的

来说，为了他人（包括未来的自己！）着想，请像使用许多可嵌套工具一样，少用、善用 `:=`。清晰通常值得多写一两行。

深度理解

·核心概念：`:=`（昵称 walrus operator，海象运算符）是“能返回值的赋值”：一条表达式里同时完成赋值 + 取用。它不做 `=` 能做到之外的新事，只是把“先赋后用”从两行压成一行。

·底层实现：`name := value` 编译为 `value` 求值后紧跟 `DUP TOP`（复制一份值）+ `STOREFAST name`，栈顶剩下的那份成为表达式结果。所以它天生只支持单一名字——下标、属性目标无法 `DUP TOP` 后再“存”同一个容器上下文，于是语法上直接禁止（这正是 `python[0] := 3.12` 报 `SyntaxError` 的原因）。

·设计原因：PEP 572（2018，Python 3.8）。Python 官方一直拒绝 C 的 `a = b = ...` 表达式化，因为 `=` 与 `==` 混淆的代价太高；`:=` 用全新、视觉醒目的符号规避了这个风险，并有意限制目标种类、禁止增强形式，防止出现 `:= + =` 这种把水搅浑的组合。

·实际问题：① `while line := f.readline():` 读文件到 EOF；② `if (n := len(a)) > 10:` 避免重复求值 `len(a)`；③ 推导式中捕获中间值：`[y for x in data if (y := f(x)) > 0]`——注意这里变量在推导式外部也可见；④ 正则 `if (m := re.search(pat, text)): use(m)`。

·初学者误区：① 不包括号直接写 `if line := f.readline() !=`

"——绑定优先级问题，`!=` 先于 `:=`，结果把布尔值赋给 `line`；②以为 `:=` 是语句——它是表达式，需要出现在表达式上下文；③在推导式/循环里过度使用导致变量泄漏与可读性崩坏——PEP 572 自己都建议"少用"；④期待它支持 `x[0] := v` 或 `x += ...` 的形式——都不存在。

代码分析

```
>>> a = 'hack' * (b := 2)          # b=2 2 a='hackhack'
>>> (python := 3.12) + 0.01        # + 3.13

>>> line = file.readline()         #
>>> if line:                        #
...     print(line)
>>> if line := file.readline():    #
...     print(line)                # line

>>> while (line := file.readline()) != '': # := !=
...     print(line)                # line

>>> [val := 'Py!', val * 2, val * 3] # val
>>> list(pow := 2 ** num for num in [2, 4, 8]) # pow=256
```

解释器如何处理：`a = 'hack' (b := 2)` 求值顺序为：加载'hack' → 进入括号，2 压栈 → DUPTOP（复制 2）→ STOREFAST b（赋走一份）→ BINARYOP 用剩下一份做乘法 → STOREFAST a。而在 `while (line := f.readline()) != stop` 中，若去掉括号，语法分析器会按优先级把表达式切为 `line := (f.readline() != stop)`——先做比较再做赋值，`line` 得到布尔值而非字符串，行为完全错位。这正是 `:=` 语法规则"鼓励括号"的体现：其优先级低于比较运算符、高于逗号。

11.11 变量命名规则 (Variable Name Rules)

英文原文

Now that we've explored assignment statements and expressions, it's time to get more formal about the use of variable names. In Python, names come into existence when you assign values to them, but there are a few rules to follow when choosing names for the subjects of your programs: Syntax: (underscore or letter) + (any number of letters, digits, or underscores). Variable names must start with an underscore or letter, which can be followed by any number of letters, digits, or underscores. `hack`, `hack`, and `Hack1` are legal names, but `1hack`, `hack$`, and `@#!` are not. Python also allows non-ASCII Unicode characters to appear in variable names, but such characters may make your code difficult to use in some contexts and are subject to a handful of rules that require deep Unicode knowledge and are too arcane to cover here. See Chapter 37's note "Unicode in variable names" for expanded coverage of this topic, and Python's language manuals for full details. Case matters: `HACK` is not the same as `hack`. Python always pays attention to case in programs, both in names you create and in reserved words. For instance, the names `X` and `x` refer to two different

variables. For portability, case also matters in the names of imported module files, even on platforms where the filesystems are case-insensitive (as is common in Windows). That way, your imports still work after programs are copied to differing platforms. Reserved words are off-limits. Names you define cannot be the same as words that mean special things in the Python language. For instance, Python will raise a syntax error if you try to use `class` as a variable name, but `klas` and `Class` work fine. Table 11-3 lists the words that are currently reserved—and hence off-limits for names of your own—in Python. Note: Python's docs call these "keywords" today, despite the longstanding and differing use of "keywords" for pass-by-name function arguments; to avoid confusion, this book uses "reserved words" for, well, reserved words. Table 11-3. Python reserved words

False await else import pass None break except in raise True class finally is return and continue for lambda try as def from nonlocal while assert del global not with async elif if or yield

In addition to Table 11-3 (and as partly leaked in Chapter 10), the words `match`, `case`, `,` and `type` are soft reserved words: they are reserved only in the context of the statement to which they belong and can be used as variables anywhere else. As you'll learn in the next chapter, the first three are part of a multiple-choice `match` statement; the latter is used by a `type` state

ment that creates type aliases, used for the type hinting discussed at the end of Chapter 6. As you can see, most of Python's reserved words are all lowercase. They are also all truly reserved—unlike names in the built-in scope that you will meet in the next part of this book, you cannot redefine reserved words by assignment. The statement `and = 1`, for instance, results in a syntax error. Besides being of mixed case, the first three entries in Table 11-3—`False`, `None`, and `True`—are somewhat unusual in meaning: they also appear in the built-in scope of Python described in Chapter 17, and they are technically names assigned to objects. They are truly reserved in all other senses, though, and cannot be used for any other purpose in your script other than that of the objects they represent. All the other reserved words are hardwired into Python's syntax and can appear only in the specific contexts for which they are intended. Furthermore, because module names in import statements become variables in your scripts, variable name constraints extend to your module filenames too. For instance, you can code files called `and.py` and `my-code.py` and can run them as top-level scripts, but you cannot import them: their names without their `.py` extensions become variables in your code on imports, and so must follow all the variable rules just outlined. Hence, reserved words are of f-limits, and dashes won't work, though underscores will. This module idea will be revisited in Part V of this

s book, where you'll find that this constraint also applies to names of "package" folders by extension.

中文翻译

既然我们已经探索了赋值语句和表达式，是时候更正式地谈谈变量名字的用法了。在 Python 中，名字在你赋值给它们时才存在，但为程序对象选择名字时要遵循一些规则：语法规则：（下划线或字母）+（任意数量的字母、数字或下划线）。变量名必须以字母或下划线开头，后面可跟任意数量的字母、数字或下划线。hack、hack、Hack1 都是合法名字，但 1hack、hack\$、@#! 不合法。Python 也允许非 ASCII 的 Unicode 字符出现在变量名中，但在某些环境中这类字符可能让你的代码难以使用，而且它们受制于一整套需要深厚 Unicode 知识、过于晦涩而无法在此展开的规则。关于此主题的扩展论述见第 37 章“变量名中的 Unicode”注记，完整细节见 Python 语言手册。大小写敏感：HACK 与 hack 不是一回事。Python 在程序中始终注意大小写，既包括你创建的名字，也包括保留字。例如名字 X 和 x 是两个不同的变量。出于可移植性考虑，导入的模块文件名字也区分大小写，即使在文件系统大小写不敏感的平台（如 Windows 常见那样）也如此。这样，程序被复制到不同平台后，你的导入仍然能正常工作。保留字（reserved word）是禁区。你定义的名字不能与 Python 语言中有特殊含义的词相同。例如，如果你试图用 class 作为变量名，Python 会抛出语法错误；但 klass 和 Class 都能正常工作。表 11-3 列出了 Python 中当前被保留——因而你自己的名字不能用——的单词。注意：Python 官方文档如今把这些词叫作 “keywords”（关键字），尽管 “keywords” 长期以来一

直有不同的用法——指按名称传参的函数实参；为避免混淆，本书用“保留字”来指这些保留词。表 11-3：Python 保留字 (False、None、True、and、as、assert、async、await、break、class、continue、def、del、elif、else、except、finally、for、from、global、if、import、in、is、lambda、nonlocal、not、or、pass、raise、return、try、while、with、yield) 除了表 11-3 (以及第 10 章泄露的一点点内容)，单词 match、case、和 type 是软保留字 (soft reserved words)：它们只在所属语句的上下文里被保留，在其他任何地方都可以用作变量。你会在下一章学到，前三个是多项选择的 match 语句的一部分；最后一个 type 由创建类型别名的 type 语句使用，服务于第 6 章末尾讨论的类型提示。如你所见，大多数 Python 保留字都是小写。它们也都被真正保留——与你在本书下一部分将遇到的、属于内置作用域 (built-in scope) 的名字不同，你不能通过赋值重新定义保留字。例如语句 `and = 1` 会导致语法错误。表 11-3 中前三项——False、None、True——除了大小写混合之外，含义有点特殊：它们也出现在第 17 章描述的 Python 内置作用域中，而且在技术上是赋给对象的名字。不过它们在所有其他意义上确实是被保留的，在你的脚本中除了代表它们所指的对象之外，不能用于任何其他目的。所有其他保留字都被硬编码进 Python 语法，只能出现在为其设定的特定上下文里。此外，因为 `import` 语句中的模块名会变成脚本里的变量，变量名约束也延伸到了你的模块文件名。例如，你可以写名为 `and.py` 或 `my-code.py` 的文件并作为顶层脚本运行，但不能导入它们：去掉 `.py` 扩展名后的名字在导入时会成为代码里的变量，因此必须遵守上面列出的所有变量规则。也就是说，保

留字是禁区，连字符（短横线）不行，而下划线可以。这个模块的概念将在本书第五部分再次涉及，到时候你会发现这条约束按扩展也适用于"包"文件夹的名字。

深度理解

·核心概念：命名三规则——起始字符、大小写敏感、避开保留字；外加软保留字这一 3.10 后的新类别。名字没有类型，只有对象有类型。

·底层实现：Python 的编译器在解析源码时，如果标识符命中保留字表就立即报 `SyntaxError`；`False/None/True` 是特殊的常量字面量（字节码为 `LOADCONST`），而软保留字 `match/case` 的作用域限定由上下文相关语法（`context-sensitive grammar`）实现——CPython 的词法分析器据其前一关键词决定是否保留。

·设计原因：动态语言"名字随意创建"意味着没有编译器帮你挡拼写错误，保留字表是语言与代码之间最后的硬性契约；软保留字则是折中——既要新增语法关键词（`match/case`）又不破坏既有代码里同名的变量。

·实际问题：模块文件名 `my-script.py` 能跑不能 `import`；IDLE 里 `class` 变量报错；跨平台部署时 `MyModule.py` 与 `mymodule.py` 冲突的文件系统大小写 bug；`type` 在 3.12 中已真成保留（`type alias` 语句）。

·初学者误区：① `HACK` 与 `hack` 完全两个变量，新手常忽而大写忽而小写导致 `NameError`；②以为 `x` 是私有变量——它只是约定，不会被 `from module import` 导入，但直接访问仍可行；③把 `match/case` 当成了嚴禁，其实

在 `match` 语句之外它们可作普通变量名。

·作者絮叨：Lutz 特意点出 `and.py` 可运行却不可导入——活生生的“命名约束连着文件系统”的教训，值得每个写多文件程序的读者记住。

代码分析

```
_hack = 1      #
hack = 2       #
Hack_1 = 3     # +
# 1_hack = 4   # SyntaxError
# class = 5    # SyntaxError
# and = 1      #
match = 6      # soft keyword match
case = 7       #
_ = 8          # REPL _ match
klass = things # class
```

解释器如何处理：语法分析阶段，分词器 (tokenizer) 遇到标识符会先查保留字表——命中则产生对应的关键字 token 而非 NAME token，后续语法规则据此报错 (`class = 5` 立即失败)。False/None/True 属于关键字 token，被映射到常量对象。而 `match` 之后若紧跟一个整数表达式，分词器会按上下文把其后的 `case` 当软保留字识别；单独一行 `match = 6` 时，`match` 就是普通 NAME。这也是 3.10 引入 soft keyword 后，老代码里叫 `match/case` 的变量仍安然无恙的原因。

11.12 旁注：Python 的弃用策略 (PYTHON'S DEPRECATION POLICIES)

英文原文

Because making new words reserved has the potential to break code widely, such changes are generally phased into the language gradually. When a change might impact existing code, Python often makes it an option and begins issuing deprecation warnings one or more releases before the mod is officially enabled. The idea is that you should have time to notice the warnings and update your code before upgrading your Python. More recently, the "soft" reserved-word category has arisen to allow new reserved words to be added with neither warnings nor breakages. This seems a tacit recognition that deprecations only go so far. Deprecation policies are not always followed, especially for major releases like 3.0 (which broke existing code wantonly), but also for newer changes deemed to be justifiable or innocuous by their promoters. In theory, changes are overseen by a steering committee charged with enforcing deprecation policies. While this helps in some cases, it is largely ineffectual in stemming the flood of opinionated morph and bloat in Python, which makes engineering reliable and durable software more challenging than it should be. Even when deprecation warnings are issued, though, programs that cannot be easily changed will break if they have been distributed in source code form to users who innocently upgrade their local Python. Those on the re

ceiving end of such flux may legitimately see deprecation warnings less as a courtesy than a bandage on a gaping wound. Warning that you're going to be rude doesn't make it OK to be rude!

中文翻译

因为把新词变成保留字可能大面积破坏代码，这类改动一般会被分阶段地逐渐引入语言。当一个改动可能影响既有代码时，Python 通常会先把它做成可选项，并在该改动正式生效之前一到多个版本发布弃用警告（deprecation warning）。其想法是：你应该有时间注意到警告、在升级 Python 之前更新代码。最近，又出现了“软”保留字这一类别，让新的保留字既能加入、又不需要警告或破坏。这似乎默认承认了：弃用政策能做的也就这么多。弃用政策并不总被遵守，尤其是像 3.0 这样的重大发布（它肆意破坏既有代码），也包括那些被推动者认为“有理或无害”的新改动。理论上，改动由一个专门负责执行弃用政策的指导委员会（steering committee）监管。虽然这在某些情况下有帮助，但对于抑制 Python 中主观倾向的变异与膨胀洪流，它基本上无济于事——这让工程化构建可靠而持久的软件变得比应有的更难。不过，即使发出了弃用警告，那些不便于修改的程序，一旦以源码形式分发、而用户又无意间升级了本机 Python，仍然会崩溃。站在这种变动接受端的人，或许有理由把弃用警告看得不像是一种礼貌，而是一块糊在巨大伤口上的创可贴。警告你“我要失礼了”，并不代表失礼就变得正当！

深度理解

·核心概念：保留字不是一夜之间增加的——Python 通过

"先警告、后启用"的节奏控制语言演进；软保留字则彻底免去警告环节。

·设计原因：语言要进化，但用户代码不能随便坏。deprecation warnings 是官方给迁移期开的"黄灯"，soft keyword 是"绿灯直接换路标"的更进一步方案。

·实际问题：升级 Python 大版本时，运行程序观察 DeprecationWarning 是工程惯例（python -W error::DeprecationWarning 强制当成错误来抓）。

·作者态度：这段带着明显的个人批评色彩——Lutz 对 3.0 大体上很不满（"肆意破坏"），认为指导委员会对语言的持续膨胀约束不力。读者应对"弃用政策"保持清醒：它不等于"永远不破坏你的代码"。

11.13 命名约定 (Naming Conventions)

英文原文

Besides these rules, there is also a set of naming conventions—rules that are not required but are followed in normal practice. For instance, because names with two leading and trailing underscores (e.g., `_name_`) generally have special meaning to the Python interpreter, you should avoid this pattern for your own names except in contexts where it is expected. Here is a list of the conventions Python follows: Names that begin with a single underscore (X) are not imported by a `from module import` statement, described in Chapter 2

3. Names that have two leading and trailing underscores (X) are system-defined names that have special meaning to the interpreter and provide implementation details in the user-defined OOP classes of Part VI. Names that begin with two underscores and do not end with two more (X) are localized ("mangled") to enclosing classes, per the discussion of pseudoprivate attributes in Chapter 31. The name that is just a single underscore () retains the result of the last expression when you are working interactively at some REPLs and is a soft keyword for a wildcard in match, as covered in Chapter 12. In addition to these Python interpreter conventions, there are various other conventions that Python programmers usually follow. For instance, some programmers distinguish parts of long names using "camelCase" (aLongName), and others use underscores (aLongname); either is completely valid according to both Python and this book. Later in the book, you'll also see that class names commonly start with an uppercase letter and module names with a lowercase letter, and that the name `self`, though not reserved, usually has a special role in classes. Moreover, in Chapter 17 we'll study another, larger category of names known as the built-ins, which are predefined but not reserved (and so can be reassigned: `open = 99` silently works, though you might occasionally wish it didn't!).

中文翻译

除了这些规则，还有一套命名约定 (naming convention) ——不是硬性要求、但实践上大家普遍遵循的规则。例如，因为带两个前导与结尾下划线的名字 (如 `name`) 通常对 Python 解释器有特殊意义，除了那些本应如此的上下文之外，你应该避免为自己的名字使用这种模式。下面是 Python 遵循的约定清单：以单个下划线开头的名字 (X) 不会被 `from module import` 语句导入，详见第 23 章。带两个前导与结尾下划线的名字 (X) 是系统定义的名字，对解释器有特殊意义，并给第六部分的用户自定义 OOP 类提供实现细节。以两个下划线开头且不以两个下划线结尾的名字 (X) 会被局部化 (英文俗称“名字改写”，name mangling) 到所在类，相关内容见第 31 章伪私有属性的讨论。仅仅一个下划线的名字 () 在交互式 REPL 环境下会保留最后一次表达式的结果，同时也是 `match` 中的通配符 `soft keyword`，见第 12 章。除了这些解释器约定的名字之外，Python 程序员通常还遵循各种其他约定。例如，有些程序员用“驼峰式” (camelCase，如 `aLongName`) 区分长名字的各部分，另一些用下划线 (如 `aLongname`)；按 Python 和本书的规定，两者都完全合法。本书后面你还会看到，类名通常以大写字母开头，模块名以小写字母开头，名字 `self` 虽然不是保留字，但在类里通常起特殊作用。此外，在第 17 章我们将研究另一类更大的名字——内置名 (built-ins)，它们是预定义但不保留的 (因此可以被重新赋值：`open = 99` 能悄悄执行，不过你可能偶尔宁愿它不行！)。

深度理解

- 核心概念：四条下划线约定——X（半私有，不进）、X（系统魔法，勿造）、X（类内名字改写）、（REPL 上次结果 / 通配符）。再加上风格约定：类名 PascalCase、模块名 lowercase、self 惯例。
- 底层实现：X 的名字改写由编译器完成——类内出现 X 会被改写为 ClassNameX，实现"pseudo-private"属性，这并非真正私有（外部改写后仍可访问）；作为 REPL 结果变量是 sys 的交互模式内置行为。
- 设计原因：Python 用约定而非强制关键字实现"私有"，符合"程序员都是成年人"的哲学：约定给你提示，但绝不阻止你。
- 实际问题：from module import 只导公共名——用开头可以轻松控制包导出面；调试交互式代码时保存上一条结果很实用。
- 初学者误区：认为 double 是"真正的私有"——它只是改名；认为所有 x 都由自己定义——造一个自己的 foo 可能在未来 Python 版本与系统名冲突，属于自找麻烦。

11.14 名字没有类型，但对象有 (Names Have No Type, but Objects Do)

英文原文

This is mostly review, but remember that it's crucial t

o keep Python's distinction between names and objects clear. As described in Chapter 6, objects have a type (e.g., integer, list) and may be mutable or not. Names (a.k.a. variables), on the other hand, are always just references to objects; they have no notion of mutability and have no associated type information, apart from the type of the object they happen to reference at a given point in time. Thus, it's OK to assign the same name to different kinds of objects at different times: >>> x = 0 # x bound to an integer object >>> x = 'Hello' # Now it's a string >>> x = [1, 2, 3] # And now it's a list Especially when we step up to functions and classes later in this book, you'll see that this generic nature of names can be a decided advantage in Python programming. In Chapter 17, you'll also learn that names live in something called a scope, which defines where they can be used; the place where you assign a name determines where it is visible.

中文翻译

这大都是复习，但请记住，厘清 Python 中名字与对象的区别至关重要。如第 6 章所述，对象有类型（例如整数、列表），并且可能是可变的或不可变的。而名字（也就是变量）永远只是对对象的引用；它们没有可变性的概念，也没有任何关联的类型信息，除了在某一刻它碰巧引用的那个对象的类型之外。因此，同一个名字在不同时间被赋给不同类型对象是完全允许的：>>> x = 0 # x 绑定到一个整数对象>>> x = 'Hello' # 现在它是个字符串>>> x = [1, 2, 3]

现在它是个列表尤其是当我们在本书后面进阶到函数和类时，你会看到名字的这种通用性在 Python 编程中是一个明显的优势。在第 17 章，你还会学到名字住在一种叫作用域（scope）的东西里，它定义了名字可以在哪里使用；你给名字赋值的位置决定了它在哪可见。

深度理解

·核心概念：这是全书最重要的心智模型之一——类型属于对象，不属于名字（动态类型 typing）。名字只是“当前时刻指向某对象的引用标签”。

·底层实现：Python 变量名不携带类型，解释器运行时才确定对象类型；CPython 中名字是作用域字典/符号表里的键，值是对对象的指针。

·设计原因：动态类型带来极大灵活性（鸭子类型、多态写函数），代价是错误在运行时才暴露——所以前面“用前必须赋值”的错误越早报越好。

·实际问题：一个函数可以接收 int、str、list 任意类型的工作，没有 C++/Java 的静态约束；但也意味着大型项目需要类型注解等工具辅助。

·初学者误区：把变量想象成“有类型的盒子”，进而困惑“为什么 x 一会儿是字符串一会儿是列表”。正确理解：x 这根标签从未变化，变的是它所贴的对象。

代码分析

```
>>> x = 0          # x " 0"x ->  int(0)
>>> x = 'Hello'    # x  int(0)
>>> x = [1, 2, 3]   # x
>>> type(x)         # ""
<class 'list'>
```

解释器如何处理：x='Hello' 时，整数字面量 0 常量化对象无人引用，会被 CPython 的引用计数在赋值瞬间置 0 并立即销毁（除非小整数有缓存）；x 则被重新 STOREF AST 指向新字符串。名字槽里始终只是“一个指针”，内容物类型随对象而变——这正是动态语言的底层写照。

11.15 表达式语句 (Expression Statements)

英文原文

In Python, you can use any expression as a statement, too—which usually means on a line by itself. Because the result of the expression won't be saved, though, it usually makes sense to do so in scripts only if the expression does something useful as a side effect. Expressions are commonly used as statements in two situations: For calls to functions and methods. Some functions do their work without returning a value. Such functions are sometimes called procedures in other languages. Because they don't return values that you might be interested in retaining, you can call these functions with expression statements. This also applies

to methods, which are just functions with an implied subject.

For printing values at the interactive prompt. As you certainly know by now, Python echoes back the results of expressions typed at the interactive command line. Technically, these are expression statements, too; they serve as a shorthand for typing print statements.

Table 11-4 lists some common expression statement forms in Python. Calls to functions and methods are coded with zero or more argument objects (really, expressions that evaluate to objects) in parentheses, after the function or method name.

Table 11-4. Common Python expression statements | Operation | Interpretation | |---|---| | hack('Py', 3.12) | Function calls | | code.hack('Py') | Method calls | | hack | Printing results in the interactive interpreter (REPL) | | print(a, b, c, sep='') | Printing operations (a special function call) | | yield x 2 | Yielding expression statements (generators) | | await producer() | Pausing for steps to finish (coroutines) | The last three entries in Table 11-4 are somewhat special cases. As you'll see later in this chapter, printing in Python is a function call usually coded as a statement by itself, and important enough to callout here. The yield and await operations on generator and coroutine functions (discussed in Chapter 20) are regularly coded as statements as well. All three, though, are really just instances of expr

essions masquerading as statements.

For instance, though you normally run a print call on a line by itself as an expression statement, it actually returns a value like any other function call—the return value is None, the default return value for functions that don't return anything meaningful (it requires an other print to reveal in output):

```
>> x = print('code') # print is a function-call expression  
>> print(x) # But is usually coded as an expression statement  
None
```

Also keep in mind that although expressions can appear as statements in Python, the converse is not true: statements cannot be used as expressions. A statement that is not also an expression must generally appear on a line all by itself, not nested in a larger syntactic structure.

For example, Python doesn't allow you to embed basic assignment statements (=) in other expressions. The rationale for this is that it avoids common coding mistakes; you can't accidentally change a variable by typing = when you really mean to use the == equality test. If you really miss this coding pattern from other languages, though, the newer and visually distinct: = named-assignment expression we met earlier works the same way with less chance of being confused with ==, and a variety of coding alternatives achieve similar goals (e.g., see while coverage in Chapter 13).

Expression Statements and In-Place Changes This brings up a common mistake in Python work, which we've encountered before, but is so pervasive that it merits another quick nag here. Expression statements are often used to run list methods that change a list in place:

```
>> L = [1, 2] >> L.append(3) # Append is an in-place change >> L [1, 2, 3]
```

It's not unusual, though, for Python newcomers to code this as an assignment statement instead:

```
>> L = L.append(4) # But append returns None, not L >> print(L) # So we lose our list! None
```

But this doesn't work: in-place change methods like `append`, `sort`, and `reverse` always change the list in place, but do not return the list they have changed; instead, they return the `None` object. Assigning such an operation's result back to the variable name loses your reference to the list (and it's probably garbage-collected in the process).

So don't do that—call in-place change operations without assigning their results. We'll revisit this phenomenon in "Common Coding Gotchas", because it can also appear in the context of some looping statements we'll explore in the chapters ahead.

中文翻译

在 Python 中，你也可以把任何表达式用作语句——通常意味着表达式单独占一行。不过，因为表达式的结果不会被保存，只有在表达式以副作用（side effect）的形式做了有用的事情时，在脚本里这样写才有意义。表达式通常在两种情况下被用作语句：调用函数和方法（calls to functions and methods）。有些函数做自己的工作是不返回值。这类函数在其他语言中有时被称为过程（procedure）。因为它们不会返回你可能想保留的值，你可以用表达式语句调用它们。这也适用于方法——方法不过是带有隐含主体的函数。在交互式提示符下打印值（printing values at the interactive prompt）。你当然已经知道，Python 会把你在交互式命令行输入的表达式的结果回显出来。严格来说，这些也是表达式语句；它们是输入 print 语句的快捷方式。表 11-4 列出了一些 Python 常见的表达式语句形式。函数与方法的调用，在函数或方法名后面的括号里写零个或多个参数对象（确切地说是求值为对象的表达式）。表 11-4：常见的 Python 表达式语句（函数调用 / 方法调用 / 在交互式解释器（REPL）中打印结果 / 打印操作（一种特殊的函数调用） / 产出表达式语句（生成器 generator） / 暂停等待步骤完成（协程 coroutine））表 11-4 中最后三项有些特殊。正如本章后面会看到的，Python 中的打印是一个函数调用，通常作为独立语句书写，重要到要在这里专门标出。对生成器和协程函数（第 20 章讨论）的 yield 和 await 操作也经常被写成语句。不过，这三者都真的只是“伪装成语句的表达式”。例如，虽然你通常把 print 调用单独写成一作为表达式语句，但它其实和其他函数调用一样会返回值

——返回的值是 `None`，也就是不返回任何有意义内容函数的默认返回值（要看到它，得再打印一次）：

```
>>> x = print('code') # print 是函数调用表达式code
>>> print(x)
# 但通常被写成表达式语句None 另外请记住，虽然表达式在 Python 中可以作为语句出现，但反过来不成立：语句不能用作表达式。一个不是表达式的语句，通常必须单独占一行，不能嵌套在更大的语法结构中。例如，Python 不允许你把基本赋值语句（=）嵌入到其他表达式中。这样设计的理由是避免常见的编码错误；你不会在本想使用 == 相等性测试时不小心敲 = 而意外改变变量。不过，如果你真的怀念其他语言里的这种编码模式，我们之前见过的、更新且视觉上明显不同的 := 命名赋值表达式能以同样的方式工作，且与 == 混淆的机会更小；另外还有各种编码替代方案能达到类似目的（例如见第 13 章的 while 讲解）。表达式语句与原地修改（expression statements and in-place changes）这引出了 Python 工作中的一个常见错误，我们之前遇到过，但它太普遍了，值得在这里再唠叨一次。表达式语句经常被用来运行原地修改列表的列表方法：
```

```
>>> L = [1, 2]
>>> L.append(3) # append 是原地修改
>>> L [1, 2, 3]
不过，Python 新手常常把它写成一个赋值语句：
```

```
>>> L = L.append(4) # 但 append 返回 None，不是 L
>>> print(L) # 所以我们丢掉了列表！None 但这不成立：append、sort、reverse 这类原地修改方法总是原地修改列表，但不返回它们修改过的列表；相反，它们返回 None 对象。把这类操作的结果赋回变量名，就丢失了你对列表的引用（列表很可能在这个过程中被垃圾回收）。所以不要那样写——调用原地修改操作时不要给结果赋值。我们将在“常见编码陷阱”（Common Coding Gotchas）中再次讨论这
```

个现象，因为它在后面章节将探索的某些循环语句上下文中也会出现。

深度理解

·核心概念：任何表达式都能当语句（单独一行），但值会被丢弃，所以只应使用“有副作用”的表达式。语句与表达式不对称：表达式可作语句，语句不可作表达式。

·底层实现：交互式 REPL 会把“表达式语句”的求值结果自动回显——这是 `sys.displayhook` 机制（把结果 str 化后打印，且存入）。脚本文件里的表达式语句则把栈顶结果直接丢弃。

·设计原因：`=` 不是表达式，就是为了杜绝 C 语言 `if (x = 5)` 这类把赋值当比较的经典 bug；Python 选择用 `:=` 在“表达式化”需求上做个有护栏的出口。

·实际问题：最常见的表达式语句就是 `print(...)`、`L.append(x)`、`d[k] = v` 之前那些“调用即做事”的方法调用，以及 `yield` 生成器产出。反例则是新手 `L = L.sort()` 的名场面：`sort` 原地排序并返回 `None`，赋回去就把列表弄丢了。

·初学者误区：① `L = L.append(4)` 后 `L` 变 `None`，列表对象可能被 GC；②在脚本里写裸的 `someexpression` 以为会像 REPL 一样回显——不会，脚本不会打印任何东西；③把 `=` 写进 `if` 或 `while` 的条件里——语法错误，且这是有意为之。

代码分析

```

>>> L = [1, 2]
>>> L.append(3)           # None
>>> L
[1, 2, 3]
>>> L = L.append(4)       # append  NoneL  None
>>> print(L)             # None
None
>>> x = print('code')     # print  Nonex = None
code

```

解释器如何处理：L.append(3) 这条表达式语句在字节码层面先 LOADFAST L、加载方法对象并调用，产生返回值（None），随后解释器直接丢弃栈顶——脚本模式下值被 POPTOP 弹出丢掉；REPL 模式下则交给 displayhook 打印。L = L.append(4) 则把 None STOREFAST 进 L，于是旧列表 [1,2,3] 引用计数归零被回收。这正是“原地修改方法不要赋回”这句格言的底层原因：方法协议约定返回 None 是为了提醒你“我改的是对象本身，结果没别的”。

11.16 print 操作 (Print Operations)

英文原文

In Python, print prints things—it's simply a programmer-friendly interface to the standard output stream. It's a function-call expression that we're calling out here because it's so pervasive and is usually coded as a statement. Technically, printing converts one or more objects to their textual representations, adds some

minor formatting, and sends the resulting text to either standard output or another file-like stream. In a bit more detail, `print` is strongly bound up with the notions of files and streams in Python: File object methods. In Chapter 9, we explored file object methods that write text (e.g., `file.write(str)`). Printing operations are similar, but more focused—whereas file write methods write strings to arbitrary files, `print` writes objects to the `stdout` stream by default, with some automatic conversion and formatting added. Unlike with file methods, there is no need to convert objects to strings when using `print` operations. Standard output stream. The standard output stream (often known as `stdout`) is simply a default place to send a program's text output. Along with the standard input and error streams, it's one of three data connections created when your script starts. The standard output stream is usually mapped to the window where you started your Python program or REPL, unless it's been redirected to a file or pipe in your operating system's shell or rerouted by your coding GUI. Because the standard output stream is available in Python as the `stdout` file object in the built-in `sys` module (i.e., `sys.stdout`), it's possible to emulate `print` with file write method calls. However, `print` is noticeably simpler to use in most roles and makes it easy to print text to other files and streams. NOTE — Blast from the past: Printing is also one of the most visible places where Python 3.X and 2.X diverge

d: it morphed from statement in 2.X to built-in function in 3.X—and broke nearly every existing Python program in the process. This book no longer covers 2.X's statement, but the legacy of 3.X's backward incompatibilities lives on as a natural brake on language changes (and in the muscle memory of Python coders who still type `print x` unconsciously!).

中文翻译

在 Python 中，`print` 就是打印东西——它只是一个对标准输出流（standard output stream）友好的程序员接口。它是一个函数调用表达式，我们在这里专门点出它，是因为它无处不在、又通常被写成语句形式。从技术上说，打印会把一个或多个对象转换成它们的文本表示，加上一点轻微格式，再把得到的文本送往标准输出或其他类似文件的流（file-like stream）。再详细一点，`print` 与 Python 中文件和流的概念紧密相连：文件对象方法（file object methods）。在第 9 章我们探索过写文本的文件对象方法（如 `file.write(str)`）。打印操作类似，但更聚焦——文件写方法把字符串写到任意文件，而 `print` 默认把对象写到 `stdout` 流，并自动加上转换和格式。与文件方法不同，使用 `print` 操作时无需先把对象转换成字符串。标准输出流（standard output stream）。标准输出流（通常叫 `stdout`）只是程序文本输出的默认去处。它与标准输入、标准错误流一起，是脚本启动时创建的三条数据连接之一。标准输出流通常被映射到你启动 Python 程序或 REPL 的那个窗口，除非它在操作系统 shell 里被重定向到文件或管道，或由你的编程 GUI 另行路由。因为标准输出流在 Python 里以 `sys` 模块中的 `s`

tdout 文件对象（即 `sys.stdout`）形式可用，所以可以用文件写方法调用模拟 `print`。不过在大多数角色中 `print` 明显更简单，而且把文本打印到其他文件和流也很容易。注意（NOTE）——来自过去的爆炸（blast from the past）：打印也是 Python 3.X 与 2.X 分歧最显眼的地方之一：它从 2.X 的语句形态变成了 3.X 的内置函数——并且在这个过程中破坏了几乎每一个既有 Python 程序。本书不再涉及 2.X 的语句形态，但 3.X 向后不兼容的遗产作为语言变化的天然刹车继续存在（也存在于那些仍会下意识敲出 `print x` 的 Python 程序员肌肉记忆里！）。

深度理解

- 核心概念：`print` = 程序员友好版的 `sys.stdout.write(str(obj) + ...)`。它是函数（有返回值 `None`），只是通常作为独立语句书写。
- 底层实现：`print` 是 C 内置函数，内部把每个对象经 `PyObjectStr`（等价 `str()`）转文本，用 `sep` 连接、`end` 结尾，然后调用目标对象的 `write` 方法（默认 `sys.stdout.write`）。`sys.stdout` 是解释器启动时打开的文件对象（连接 OS 的文件描述符 1）。
- 设计原因：3.X 把 `print` 从语句变成函数，是为了获得语句无法提供的灵活性——`sep/end/file/flush` 这些行为都成了可配置参数，还允许把 `print` 本身当作对象传递（`printer = print`）。
- 实际问题：日志落盘（`file=`）、进度条无换行（`end=""`）、对齐输出（`sep=','`）、强制刷新（`flush=True` 用于实时日志）、给 `sys.stdout` 换对象实现全局重定向。

·初学者误区：以为 `print` 是语句——它是函数；以为 `print` 和 `str()` 必须配合 `f-string` 才专业——多数简单打印直接逗号分隔即可；把 `print` 当关键字无法 `print = ...?` 可以重新赋值（`built-in` 可覆盖），只是别这么干。

11.17 `print` 函数 (The Print Function)

英文原文

Strictly speaking, printing is not a separate statement form. Instead, it is simply an instance of the expression statement we studied in the preceding section. The `print` built-in function is normally called on a line of its own, because it doesn't return any value we care about (technically, it returns `None`, per the preceding section).

Because it is a normal function, though, it can use standard function-call syntax (including keyword arguments for special operation modes) and may be both passed around as an object and reassigned to a different implementation.

Call format Syntactically, calls to the `print` function have the following form:

```
print([object, ...][, sep=' '][, end='\n'][, file=sys.stdout][, flush=False])
```

In this notation, items in square brackets are optional and may be omitted in a given call, and values after `=` give keyword-argument defaults. In

English, this built-in function prints the textual representation of one or more objects, separated by the string `sep` and followed by the string `end`, to the stream `file`, flushing buffered output or not per `flush`.

The `sep`, `end`, `file`, and `flush` parts, if present, must be given as keyword arguments—that is, you must use a `name=value` syntax to pass the arguments by name instead of position. Keyword arguments are covered in depth in Chapter 18, but they're straightforward to use.

The keyword arguments sent to this call may appear in any left-to-right order following the objects to be printed, and they control the print operation: - `sep` is a string inserted between each object's text, which defaults to a single space if not passed.

Passing an empty string suppresses separators altogether. - `end` is a string added at the end of the printed text, which defaults to a `\n` newline character if not passed.

Passing an empty string avoids dropping down to the next output line at the end of the printed text—the next print will keep adding to the end of the current output line. - `file` specifies the file, standard stream, or other file-like object to which the text will be sent.

It defaults to the `sys.stdout` standard output stream if not passed, but any object with a file-like `write(string)` method may be passed, including real file objects th

at have already been opened for output to external files. - flush defaults to False. It allows prints to mandate that their text be flushed through the output stream immediately to any waiting recipients.

Normally, whether printed output is buffered in memory or not is determined by the file object; passing a true value to flush forcibly flushes the stream.

The textual representation of each object to be printed is obtained by passing the object to the `str` built-in call (or its equivalent inside Python); as we've seen, this built-in returns a "user friendly" display string for any object. With no arguments at all, the `print` function simply prints a newline character to the standard output stream, which usually displays a blank line.

The `print` function in action Printing is probably simpler than its full details may imply. To illustrate, let's run some quick examples in the REPL. The following prints a variety of object types to the default standard output stream, with the default separator and end-of-line formatting added (these are the defaults because they are the most common use case):

```
>> print() # Display a blank line >> x='python'>> y=3.12 >> z=['!p6e'] >> print(x, y, z) # Print objects per defaults python 3.12 ['!p6e']
```

There's no need to convert objects to strings here, as would be required for file write methods. By default, `print` calls add a space between the objects printed.

To suppress this, send an empty string to the `sep` keyword argument, or send an alternative separator of your choosing:

```
>> print(x, y, z, sep='') # Suppress separator python
3.12['lp6e'] >> print(x, y, z, sep=',') # Custom separator
python, 3.12, ['lp6e']
```

Also by default, `print` adds an end-of-line character to terminate the output line. You can suppress this and avoid the line break altogether by passing an empty string to the `end` keyword argument, or you can pass a different terminator of your own including a `\n` character to break the line manually if desired (the second of the following is two statements on one line, separated by a semicolon to demo the effect of custom terminators in the REPL):

```
>> print(x, y, z, end='') # Suppress line break (see >>
python 3.12 ['lp6e']>>> >> print(x, y, z, end='');
print(x, y, z) # Two prints, same output line python 3.
12 ['lp6e']python 3.12 ['lp6e'] >> print(x, y, z, end='...
\n') # Custom line end python 3.12 ['lp6e']...
```

You can also combine keyword arguments to specify both separators and end-of-line strings—they may appear in any order but must appear after all the objects being printed:

```
>> print(x, y, z, sep='...', end='!\n') # Multiple keywords
python...3.12...['lp6e']!
```

```
>> print(x, y, z, end='!\n', sep='...') # Order doesn't matter
python...3.12...['!p6e']!
```

Here is how the `file` keyword argument is used—it directs the printed text to an open output file or other compatible object for the duration of the single print (this is really a form of stream redirection, a topic we will revisit later in this section):

```
>> print(x, y, z, sep='...', file=open('data.txt','w')) # Print to a file
>> print(x, y, z) # Back to stdout
python 3.12 ['!p6e']
>> print(open('data.txt').read()) # Display file text
python...3.12...['!p6e']
```

Finally, keep in mind that the separator and end-of-line options provided by print operations are just conveniences. If you need to display more specific formatting, don't print this way. Instead, build up a more custom string either ahead of time or within the print itself, using the string tools we mastered in Chapter 7—and print the string all at once. String formatting expressions and f-strings, for example, were designed for this sort of job:

```
>> text = '%s: %-.4f, %05d' % (x, y, int(z[0][-2]))
>> print(text)
python: 3.1200, 00006
>> print(f'{x}: {y:~.4f}, {int(z[0][-2]):05d}')
python: 3.1200, 00006
```

On the other hand, there seems to be a misconception that f-strings are somehow required for print, and they crop up needlessly. In truth, f-strings are overkill if you're simply trying to print objects separated by s

paces:

```
>> a, b, c, = 11, 3.14, 'hack' >> print(f'{a} {b} {c}') 11 3
.14 hack >> print(a, b, c) 11 3.14 hack
```

As usual, don't use a sledgehammer to drive every nail!

中文翻译

严格来说，打印并不是一种独立的语句形式。它只是上一节学过的表达式语句的一个实例。内置函数 `print` 通常单独占一行调用，因为它不返回任何我们在意的值（严格说它返回 `None`，如上一节所述）。但因为它是一个普通函数，所以可以使用标准的函数调用语法（包括用于特殊操作模式的关键字参数），并且可以作为一个对象被传递，也可以被重新赋值为不同的实现。调用格式（call format）从语法上说，对 `print` 函数的调用具有如下形式：`print([object, ...], sep='', end='\n', file=sys.stdout, flush=False)` 这种记法中，方括号里的项是可选的、可在具体调用中省略，`=` 后的值给出关键字参数的默认值。用大白话说：这个内置函数打印一个或多个对象的文本表示，用字符串 `sep` 分隔、后面跟字符串 `end`，送往流 `file`，并按 `flush` 决定是否冲刷（flush）缓冲输出。`sep`、`end`、`file`、`flush` 这些部分，如果存在，必须用关键字参数（keyword argument）给出——也就是说，你必须用 `name=value` 语法按名字传参而不是按位置。关键字参数在第 18 章有深入讲解，但用起来很直接。传给这个调用的关键字参数可以按任意左右顺序出现在要打印的对象之后，它们控制着 `print` 操作：

- `sep` 是插入在每一个对象文本之间的字符串，不传时默认为一个空格。传空字符串则完全取消分隔符。
- `end` 是加在打印文

本末尾的字符串，不传时默认为换行符 `\n`。传空字符串可避免在文本末尾换行——下一个 `print` 会继续添加到当前输出行末尾。- `file` 指定文本被送往的文件、标准流或其他文件类对象。不传时默认为 `sys.stdout` 标准输出流，但任何带文件类 `write(string)` 方法的对象都可以传，包括已经打开用于输出到外部文件的真实文件对象。- `flush` 默认为 `False`。它允许 `print` 强制文本立即穿过输出流到达任何等待的接收者。通常，打印输出是否在内存中缓冲由 `file` 对象决定；给 `flush` 传真值会强制冲刷流。每个待打印对象的文本表示，是把对象传给内置 `str` 调用（或 Python 内部的等价实现）得到的；正如我们所见，这个内置函数对任何对象都返回“用户友好”的显示字符串。完全不传参数时，`print` 函数只是向标准输出流打印一个换行符，通常显示为一个空行。

print 函数实战 (the print function in action) 打印可能比它全部细节给人的印象更简单。为了说明，我们在 REPL 里快速跑几个例子。下面把各种对象类型打印到默认的标准输出流，并加上默认的分隔符和行尾格式（它们之所以是默认值，正是因为这是最常见的用例）：

```
>>> print() # 显示一个空行
>>> x='python'>>> y=3.12>>> z=['lp6e']
>>> print(x,y,z) # 按默认设置打印对象
python 3.12 ['lp6e']
```

这里不需要像文件写方法那样把对象转成字符串。默认情况下，`print` 调用会在打印的对象之间加一个空格。要抑制它，就给 `sep` 关键字参数传一个空字符串，或传一个你自选的替代分隔符：

```
>>> print(x,y,z,sep='') # 抑制分隔符
python3.12['lp6e']
>>> print(x,y,z,sep=',') # 自定义分隔符
python, 3.12, ['lp6e']
```

同样地，默认情况下 `print` 会加一个行尾字符来结束输出行。你可以给 `end` 关键字参数传空字符串来抑制它、完全避免换行；或者传一个你

自己的不同结尾符，包括按需用 `\n` 手动断行（下面第二个例子是一行里的两条语句，用分号隔开，以便在 REPL 里演示自定义结尾符的效果）：

```
>>> print(x, y, z, end='') # 抑制换行（注意 >>> 紧跟在输出后）python 3.12 ['lp6e']>>> >>> print(x, y, z, end=''); print(x, y, z) # 两次打印，同一输出行python 3.12 ['lp6e']python 3.12 ['lp6e']>>> print(x, y, z, end='...\n') # 自定义行尾python 3.12 ['lp6e']... 你还可以组合关键字参数同时指定分隔符和行尾字符串——它们可以任意顺序出现，但必须在所有被打印对象之后：
```

```
>>> print(x, y, z, sep='...', end='!\n') # 多个关键字python...3.12...['lp6e']!>>> print(x, y, z, end='!\n', sep='...') # 顺序无所谓python...3.12...['lp6e']! 下面是 file 关键字参数的用法——它把打印文本导向一个已打开的输出文件或其他兼容对象，作用范围仅限这一次 print（这其实是一种流重定向形式，本节后面还会再谈）：
```

```
>>> print(x, y, z, sep='...', file=open('data.txt','w')) # 打印到文件>>> print(x, y, z) # 回到 stdout python 3.12 ['lp6e']>>> print(open('data.txt').read()) # 显示文件内容python...3.12...['lp6e'] 最后请记住，print 操作提供的分隔符和行尾选项只是便利。如果你需要显示更具体的格式，不要这样打印。相反，用第 7 章掌握的字符串工具预先（或在 print 内部）构建一个更定制化的字符串——然后一次性打印它。字符串格式化表达式和 f-string，例如，就是为这类工作设计的：
```

```
>>> text = '%s: %-.4f, %05d' % (x, y, int(z[0][-2]))>>> print(text) python: 3.1200, 00006>>> print(f'{x}: {y:~0.4f}, {int(z[0][-2]):05d}') python: 3.1200, 00006 另一方面，似乎有一种误解：f-string 某种程度上是 print 的必需品，于是它们被不必要地大量出现。事实
```

上，如果你只是想打印被空格分隔的对象，f-string 是杀鸡用牛刀：`>>> a, b, c, = 11, 3.14, 'hack'>>> print(f'{a} {b} {c}')` 11 3.14 hack `>>> print(a, b, c)` 11 3.14 hack 老规矩：别用大锤子钉每一颗钉子！

深度理解

·核心概念：print 是函数，签名 `print(objects, sep=",", end="\n", file=sys.stdout, flush=False)`。四个关键字参数各司其职；所有对象先经 `str()` 转换。

·底层实现：print 在 CPython 里是 `builtins.print`（C 实现），接收任意数量的位置参数 + 四个关键字；内部逐个调用 `PyObjectStr`，`sep/end` 之间拼接后调用 `file.write(text)`。`file` 参数让 print 本质成为“对任意 file-like 对象的 write 调用”的语法糖。

·设计原因：函数化的 print（PEP 3105）使输出可定制、可替换、可传递；关键字参数强制用 `name=value` 传参，避免了 `print(x, y, sep, end)` 这样按位置传关键参数的混乱。

·实际问题：进度条 `print('.', end="", flush=True)`；多行日志合并 `print(..., end="")`；写文件与屏幕双输出（见下文重定向）；性能上大量小 print 会因默认行缓冲而频繁系统调用，需要时可用 `file=open(..., buffering=...)` 或一次构造大字符串。

·初学者误区：①`print(x, y, z, sep=',')` 中 `sep` 等必须放在所有对象之后，不能 `print(sep=',', x)`；②print 返回值是 `None`；③以为 `end=""` 会“删除换行”——它是“不追加换

行", 两者效果在"之后还要写文件/别的输出"时不同; ④ 拿 f-string 包一层 `print(f'{a} {b} {c}')` 完全多余——直接 `print(a, b, c)`。

代码分析

```
>>> print()                # ->
>>> print(x, y, z)          # sep=' 'end='\n'python 3.12 ...
>>> print(x, y, z, sep='')   # python3.12['lp6e']
>>> print(x, y, z, sep=', ') # python, 3.12, ['lp6e']
>>> print(x, y, z, end='')    #
>>> print(x, y, z, end='...\n') # python 3.12 ['lp6e']...
>>> print(x, y, z, sep='...', end='!\n') # python...3.12....
>>> print(1, 2, 3, file=open('log.txt', 'w')) #
>>> print(f'{x}: {y:-0.4f}, {int(z[0][-2]):05d}') # f-strin...
```

解释器如何处理: 每次 `print(...)` 都完整走一遍: 对每个位置参数求值 → `str()` 化 → 用 `sep` 连接成整体文本 → 追加 `end` → 调 `file.write(text)` (默认 `sys.stdout.write`) → 若 `flush` 为真则调用流的 `flush` 方法。注意 `file=open('log.txt','w')` 这个打开的文件对象只在这一条语句里使用, 语句结束后文件仍在内存里打开——书中示例为求简洁未关闭, 真实代码应把文件对象存起来并用 `with` 或显式 `close()` 管理。

11.18 打印流的重定向 (Print Stream Redirection)

英文原文

As we've seen, `print` sends text to the standard output stream by default. However, it's often useful to send it elsewhere—to a text file, for example, to save results for later use or testing purposes. Although such redirection can usually be accomplished in shells outside Python itself (with syntax like `> file`, per "Command-Line Usage Variations"), this is not a Python tool, and it's just as easy to redirect a script's streams from Python code.

The Python "hello world" program Let's start off with the usual (and largely pointless) language benchmark—the "hello world" program. To print a "hello world" message in Python, simply print the string with `print`:

```
>> print('hello world') # Print a string object hello world
```

Really, because expression results are echoed on the interactive command line, you often don't even need to use a `print` statement there—simply type the expressions you'd like to have printed, and their results are echoed back:

```
>> 'hello world' # Interactive echoes 'hello world'
```

This code isn't exactly an earth-shattering feat of software mastery, but it serves to illustrate printing behavior. Technically, though, the `print` operation is just an ergonomic feature of Python—it provides a simple

e interface to the `sys.stdout` object, with a bit of default formatting.

In fact, if you enjoy working harder than you must (or are pining for your Java coding days) you can also code print operations this way, which omits the write call's return value for space, as in Chapter 9:

```
>> import sys # Printing the hard way >> sys.stdout
.write('hello world\n') hello world
```

This code explicitly calls the write method of `sys.stdout`—an attribute preset when Python starts up to an open file object connected to the output stream. The print operation hides most of those details, providing a simple tool for simple printing tasks.

Manual stream redirection So, why bother learning the hard way to print? The `sys.stdout` equivalent to print turns out to be the basis of a common technique in Python. In general, print and `sys.stdout` are directly related as follows. This statement:

print(X, Y) is equivalent to the longer:

```
import sys sys.stdout.write(str(X) + " + str(Y) + '\n')
```

which manually performs a string conversion with `str`, adds a separator and newline with `+`, and calls the output stream's write method. That is, this is what the print does. Which would you rather code?

Obviously, the long form isn't all that useful for printing by itself. However, it is useful to know that this is

exactly what print operations do because it is possible to reassign `sys.stdout` to something different from the standard output stream. In other words, this equivalence provides a way of making your print operations send their text to other places. For example:

```
import sys
sys.stdout = open('log.txt','a') # Redirects
prints to a file ... print(x, y, x) # Now printed text shows up in log.txt
Here, we reset sys.stdout to a manually opened file named log.txt, located in the script's working directory (CWD) and opened in 'a' append mode (so we add to its current content). After the reset, every print operation anywhere in the program will write its text to the end of the file log.txt instead of to the original output stream. The print operations are happy to keep calling sys.stdout's write method, no matter what sys.stdout happens to refer to.
```

And because there is just one `sys` module in your program (technically, process), assigning `sys.stdout` this way will redirect every print anywhere in your program.

In fact, as the sidebar "Why You Will Care: `print` and `stdout`" will explain, you can even reset `sys.stdout` to an object that isn't a file at all, as long as it has the expected interface: a method named `write` to receive the printed text-string argument. When that object is a class, printed text can be routed and processed arbitrarily per a `write` method you code yourself.

This trick of resetting the output stream might be more useful for programs originally coded with print statements. If you know that output should go to a file to begin with, you can always call file write methods instead. To redirect the output of a print-based program, though, resetting `sys.stdout` provides a convenient alternative to changing every print statement or using system shell-based redirection syntax, which may be above users' pay grades.

In other roles, streams may be reset to objects that display them in pop-up windows in GUIs, colorize them in IDEs like IDLE, and so on. It's a general technique.

Automatic stream redirection Although redirecting printed text by assigning `sys.stdout` is a useful tool, a potential problem with the last section's code is that there is no direct way to restore the original output stream should you need to switch back after printing to a file. Because `sys.stdout` is just a normal file object, though, you can always save it and restore it if needed:

```
>> import sys >> temp = sys.stdout # Save for restoring later
>> sys.stdout = open('log.txt','a') # Redirect prints to a file
>> print('lp6e was here') # Prints go to file, not here
>> print(1, 2, 3) >> sys.stdout.close() # Flush output to disk
>> sys.stdout = temp # Restore original stream
>> print('back in the REPL') # Prints show up here again back in the REPL
>> print(open('l
```

```
og.txt').read()) # Result of earlier prints lp6e was here  
1 2 3
```

As you can see, though, manual saving and restoring of the original output stream like this involves extra juggling work. Because this crops up fairly often, a print extension is available to make it unnecessary.

As introduced earlier, the file keyword allows a single print call to send its text to the write method of a file (or file-like object), without actually resetting sys.stdout. Because the redirection is temporary, normal print calls keep printing to the original output stream. For example, the following abstract snippet again sends printed text to a file named log.txt:

```
log = open('log.txt','a') print(x, y, z, file=log) # Print to  
a file-like object print(a, b, c) # Print to original stdout  
These redirected forms of print are handy if you need to  
print to both files and the standard output stream in the  
same program. If you use these forms, however, be sure to  
give them a file object (or an object that has the same  
write method as a file object), not a file's name string.  
Here is the technique in action live:
```

```
>> log = open('log2.txt','w') >> print(1, 2, 3, file=log)  
>> print(4, 5, 6, file=log) >> log.close() >> print(7,  
8, 9) 7 8 9 >> print(open('log2.txt').read()) 1 2 3 4 5 6
```

These extended forms of print are also commonly used to print error messages to the standard error stream.

m, available to your script as the preopened file object `sys.stderr`. You can either use its file write methods and format the output manually, or print with redirection syntax:

```
>> import sys >> sys.stderr.write(('Bad!' 8) + '\n') Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!
```

```
>> print('Bad!' 8, file=sys.stderr) Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!
```

Now that you know all about print redirections, the equivalence between printing and file write methods should be clear. The following demo prints both ways, then redirects the output to an external file to verify that the same text is printed (on Unix, you won't get the `\r` added to newlines here on Windows, and write results are back here):

```
>> import sys >> X, Y = 1, 2 >> print(X, Y) # Print: the easy way 1 2 >> sys.stdout.write(str(X) + " + str(Y) + '\n') # Print: the hard way 1 2 4 >> print(X, Y, file=open('temp1','w')) # Redirect text to file >> open('temp2','w').write(str(X) + " + str(Y) + '\n') # Send to file manually 4 >> print(open('temp1','rb').read()) # Binary mode for bytes b'1 2\r\n' >> print(open('temp2','rb').read()) b'1 2\r\n'
```

As you can see, unless you happen to enjoy typing, print operations are often the best option for displaying text. For another example of the equivalence between prints and file writes, watch for a print function e

mulation example in Chapter 18; it uses the coding patterns here to provide a print equivalent that can be customized.

中文翻译

如我们所见，`print` 默认把文本送到标准输出流。不过，把它送到别处常常很有用——例如送到文本文件，把结果保存下来以备后用或测试。虽然这种重定向通常可以在 Python 之外的系统 shell 里完成（用 `> file` 这样的语法，见“命令行用法变体”），但那不是 Python 工具，而从 Python 代码里重定向脚本的流同样容易。Python 的“hello world”程序（the Python “hello world” program）让我们从那个常规的（而且基本没什么用的）语言基准——“hello world”程序开始。要在 Python 中打印一条“hello world”消息，直接用 `print` 打印字符串即可：`>>> print('hello world')` # 打印一个字符串对象 `hello world` 真的，因为交互式命令行会回显表达式结果，你甚至常常不需要在那里用 `print` 语句——直接输入你想打印的表达式，结果就会回显出来：`>>>'hello world'` # 交互式回显 `'hello world'` 这段代码算不上什么惊天动地的软件功力，但它足以说明打印行为。不过严格来说，`print` 操作只是 Python 的一个人体工学（ergonomic）特性——它给 `sys.stdout` 对象提供一个简单接口，外加一点默认格式。事实上，如果你乐于做比必要更多的功（或者怀念你的 Java 编码岁月），也可以像第 9 章那样这样写 `print` 操作，省略 `write` 调用的返回值以省空间：`>>> import sys # 费劲的打印方式>>> sys.stdout.write('hello world\n') hello world` 这段代码显式调用了 `sys.stdout` 的 `write` 方法——这是 Python 启动时预设好的

属性，指向一个连接到输出流的已打开文件对象。print 操作隐藏了这些细节，为简单的打印任务提供一个简单工具。手动流重定向（manual stream redirection）那么，为什么还要费心学习这种费劲的打印方式？sys.stdout 与 print 的等价关系正是 Python 中一项常用技术的基础。总的来说，print 与 sys.stdout 直接相关如下。这条语句：print(X, Y) 等价于更长的：import sys sys.stdout.write(str(X) + str(Y) + '\n') 后者手动用 str 做字符串转换、用 + 加分隔符和换行、再调用输出流的 write 方法。也就是说，这就是 print 所做的事。你更愿意写哪种？显然，长形式本身对打印来说没那么有用。但知道 print 操作做的正是这些事是有用的，因为有可能把 sys.stdout 重新赋值为与标准输出流不同的东西。换句话说，这个等价关系提供了一种让你的 print 操作把文本送到别处的方法。例如：import sys sys.stdout = open('log.txt','a') # 把打印重定向到文件... print(x, y, x) # 现在打印的文本出现在 log.txt 里这里，我们把 sys.stdout 重置为手动打开的文件 log.txt，它位于脚本的工作目录（CWD），以'a'追加模式打开（这样是往现有内容上追加）。重置之后，程序中任何地方的每个 print 操作都会把文本写到 log.txt 文件末尾，而不是原来的输出流。无论 sys.stdout 碰巧指向什么，print 操作都乐意继续调用它的 write 方法。而且因为你的程序（确切地说，进程）里只有一个 sys 模块，这样给 sys.stdout 赋值会把程序中任何地方的每个 print 都重定向。事实上，正如旁注"Why You Will Care: print and stdout"将要解释的，你甚至可以把它重置为一个根本不是文件的对象，只要它具备预期的接口：一个名为 write、接收打印文本字符串参数的方法。当那个对象是一个类时，打印文本可以根据你自己编写

te 方法任意路由和处理。重置输出流这个小花招，对最初用 print 语句编写的程序更有用。如果你一开始就知道输出应该进文件，总是可以调用文件写方法。但要重定向一个基于 print 的程序的输出，重置 sys.stdout 为"改每个 print 语句"或"使用系统 shell 的重定向语法"（可能超出用户的水平）提供了一种方便替代方案。在其他角色中，流可以被重置为在 GUI 弹出窗口中显示它们、在 IDLE 这样的 IDE 中给它们上色等对象。这是一种通用技术。自动流重定向（automatic stream redirection）虽然通过赋值 sys.stdout 来重定向打印文本是个有用工具，但上一节代码的一个潜在问题是：如果在打印到文件之后需要切回原始输出流，没有直接的方法恢复它。不过，因为 sys.stdout 只是一个普通文件对象，你总可以保存它、需要时再恢复：

```
>>> import sys
>>> temp = sys.stdout # 保存下来供以后恢复
>>> sys.stdout = open('log.txt','a') # 把打印重定向到文件
>>> print('lp6e was here') # 打印进入文件，不在此显示
>>> print(1, 2, 3)
>>> sys.stdout.close() # 冲刷输出到磁盘
>>> sys.stdout = temp # 恢复原始流
>>> print('back in the REPL') # 打印又在这里显示了back in the REPL
>>> print(open('log.txt').read()) # 之前打印的结果lp6e was here 1 2 3
```

不过如你所见，像这样手动保存和恢复原始输出流涉及额外的杂耍工作。因为这种情况相当常见，一个 print 扩展让这一切变得不再必要。正如前面介绍的，file 关键字允许单次 print 调用把文本送到某个文件（或文件类对象）的 write 方法，而无需真正重置 sys.stdout。因为重定向是临时的，正常的 print 调用继续打印到原始输出流。例如，下面的抽象片段再次把打印文本送到名为 log.txt 的文件：

```
log = open('log.txt','a')
print(x, y, z, fil
```

e=log) # 打印到文件类对象 print(a, b, c) # 打印到原始 stdout 如果你需要在同一个程序里既打印到文件又打印到标准输出流, print 的这些重定向形式很方便。不过, 使用这些形式时, 请确保传给它一个文件对象 (或一个与文件对象有相同 write 方法的对象), 而不是文件名字符串。下面是该技术的现场演示:

```
>>> log = open('log2.txt','w') >>
> print(1, 2, 3, file=log) >>> print(4, 5, 6, file=log) >
>> log.close() >>> print(7, 8, 9) 7 8 9 >>> print(open('log2.txt').read()) 1 2 3 4 5 6
```

print 的这些扩展形式也常用于把错误消息打印到标准错误流 (standard error stream), 它作为预打开的文件对象 sys.stderr 对你的脚本可用。你可以用它的文件 write 方法并手动格式化输出, 或者用重定向语法打印:

```
>>> import sys >>> sys.stderr.write(('Bad!' 8) + '\n') Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!
>>> print('Bad!' 8, file=sys.stderr) Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!
```

既然你已经全面了解了 print 重定向, 打印与文件 write 方法之间的等价关系应该很清楚了。下面这个演示用两种方式打印, 然后把输出重定向到外部文件以验证打印了相同文本 (在 Unix 上, 你不会在这里看到 Windows 给换行加的 \r, 而且 write 的结果会回到这里):

```
>>> import sys >>> X, Y = 1, 2 >>> print(X, Y) # print: 省事的方式 1 2
>>> sys.stdout.write(str(X) + " " + str(Y) + '\n') # print: 费劲的方式 1 2 4
>>> print(X, Y, file=open('temp1','w')) # 把文本重定向到文件
>>> open('temp2','w').write(str(X) + " " + str(Y) + '\n') # 手动发送到文件 4
>>> print(open('temp1','rb').read()) # 二进制模式读字节 b'1 2\r\n'
>>> print(open('temp2','rb').read()) b'1 2\r\n'
```

如你所见, 除非你恰好喜欢打字, print 操作往

往是显示文本的最佳选择。关于打印与文件写入等价的另一个例子，请留意第 18 章的一个 `print` 函数模拟示例；它利用这里的编码模式，提供一个可定制的 `print` 等价物。

深度理解

·核心概念：重定向上有三层工具，由局部到全局：①单次 `print(..., file=obj)`；②全局替换 `sys.stdout = open(...)`；③shell 层 `> file`。核心洞察是鸭子类型：`print` 不关心 `sys.stdout` 是谁，只要有 `write(str)` 方法就行。

·底层实现：`sys.stdout` 是解释器启动时创建的文本流对象（包装 OS 文件描述符 1，行缓冲）；`sys.stderr` 包装 fd 2（无缓冲，立即输出）。`print` 内部 `file.write(...)`；`sys.stdout = open('log.txt','a')` 之后，`print` 照样调用新的 `write`——绑定是“运行时查属性”，不是编译期固定。

·设计原因：`print` 与文件写的等价关系让“重定向”成为 Python 内置能力，无需侵入每个 `print` 调用；GUI/IDE 也能通过替换 `stdout` 对象接走全部输出。

·实际问题：日志系统把 `print` 重定向进文件；`contextlib.redirectstdout(open(f))` 是标准库对这个技巧的封装；错误信息用 `file=sys.stderr` 打印，避免与正常输出混流、也便于 shell 分别重定向 `2>`。

·初学者误区：①`print(x, file='log.txt')`——传的是文件名字符串，`str` 没有 `write` 方法，报 `AttributeError`；必须先 `open`；②重定向后忘了 `sys.stdout.close()` 导致缓冲未落盘或文件句柄泄漏；③不知道 `sys.stdout`（启动时的原始流）存在，恢复时手动保存即可；④误以为 `file=` 会

持久改变输出目标——它只影响这一次调用。

代码分析

```
>>> print(X, Y)                                # print
>>> sys.stdout.write(str(X) + ' ' + str(Y) + '\n') # + ...

import sys
sys.stdout = open('log.txt', 'a')               # print
print(x, y, x)                                  # log.txt

temp = sys.stdout                               #
sys.stdout = open('log.txt', 'a')               #
print('lp6e was here'); print(1, 2, 3)          #
sys.stdout.close()                             #
sys.stdout = temp                               #

log = open('log2.txt', 'w')                     #
print(1, 2, 3, file=log)                        #
print(7, 8, 9)                                  # stdout
print('Bad!' * 8, file=sys.stderr)              # stderr
```

解释器如何处理：print 的 C 实现先收集所有对象、str() 化、按 sep 连接、追加 end，然后做一次 file.write(text) 调用——这里的 file 是当前求值的 sys.stdout 属性（每次 print 都会重新查 sys.stdout 属性！），所以中途替换 sys.stdout 会立刻影响所有后续 print。write 返回写入的字符数（如 4），print 会丢弃该返回值；而脚本结束后 Python 会关闭所有打开的文件（CPython 关闭时刷新缓冲），但显式 close() 仍是良好实践——尤其对重定向场景，不 flush 可能丢失最后一段日志。Windows 上文本模式下写 \n 会按惯例转换为 \r\n，因此用 'rb' 读回时看到 b'1 2\r\n'，这解释了书中输出里的 \r。

11.19 章节总结 (Chapter Summary)

英文原文

In this chapter, we began our in-depth look at Python statements by exploring assignments, expressions, and print operations. Although these are generally simple to use, they have some alternative forms that, while optional, are often convenient in practice—for example, augmented and named assignments, as well as the redirection form of print operations, allow us to avoid some manual coding work. Along the way, we also studied the syntax of variable names, stream redirection techniques, and a variety of common mistakes to avoid, such as assigning the result of an append method call back to a variable. In the next chapter, we'll continue our statement tour by filling in details about the if statement, Python's main selection tool; there, we'll also revisit Python's syntax model in more depth and look at the behavior of Boolean expressions, as well as the match multiple-choice statement. Before we move on, though, the end-of-chapter quiz will test your knowledge of what you've learned here.

中文翻译

本章，我们通过探索赋值、表达式和 print 操作，开始了对 Python 语句的深入审视。虽然这些语句通常用起来很简单

，但它们有一些可选的形式，在实践中往往很方便——例如增强赋值和命名赋值，以及 `print` 操作的重定向形式，让我们避免了一些手工编码工作。一路上，我们还研究了变量名的语法、流重定向技术，以及多种要避免的常见错误，例如把 `append` 方法调用的结果赋回给变量。下一章，我们将继续语句之旅，补全 `if` 语句——Python 的主要选择工具——的细节；在那里，我们还会更深入地回顾 Python 的语法模型，看看布尔表达式的行为，以及 `match` 多项选择语句。不过在继续之前，章末测验会检验你对本章所学内容的掌握情况。

11.20 测验：检验你的知识 (Test Your Knowledge: Quiz)

英文原文

1. Name three ways that you can assign three variables to the same value. 2. What's dangerous about assigning three variables to a mutable object? 3. What's wrong with saying `L = L.sort()`? 4. How might you use the `print` operation to send text to an external file?

中文翻译

1. 说出三种把三个变量赋为同一个值的方法。2. 把三个变量赋给一个可变对象，有什么危险？3. 写 `L = L.sort()` 有什么问题？4. 你如何用 `print` 操作把文本发送到外部文件？

11.21 测验答案 (Test Your Knowledge: Answers)

英文原文

1. You can use multiple-target assignments (`A = B = C = 0`), sequence assignment (`A, B, C = 0, 0, 0`), or multiple assignment statements on three separate lines (`A = 0`, `B = 0`, and `C = 0`). With the latter technique, as introduced in Chapter 10, you can also string the three separate statements together on the same line by separating them with semicolons (`A = 0; B = 0; C = 0`).

2. If you assign them this way: `A = B = C = []`, then all three names reference the same object, so changing it in place from one (e.g., `A.append(99)`) will affect the others. This is true only for in-place changes to mutable objects like lists and dictionaries; for immutable objects such as numbers and strings, this issue is irrelevant because they can never be changed in place.

3. The list sort method is like `append` in that it makes an in-place change to the subject list—it returns `None`, not the list it changes. The assignment back to `L` sets `L` to `None`, not to the sorted list. As discussed both earlier and later in this book (e.g., Chapter 8), a newer built-in function, `sorted`, sorts any sequence and returns a new list with the sorting result; because this is not an in-place change, its result can be safely and meaningfully assigned to a name.

4. To print to a file fo

r a single print operation, you can use the `print(X, file = F)` call form, or assign `sys.stdout` to a manually opened file before the print and restore the original after if needed. You can also redirect all of a program's printed text to a file with special syntax in the system shell like `> file`, but this is outside Python's scope.

中文翻译

1. 你可以用多重目标赋值 (`A = B = C = 0`)、序列赋值 (`A, B, C = 0, 0, 0`)，或三行独立的赋值语句 (`A = 0`、`B = 0`、`C = 0`)。用最后一种技巧时，如第 10 章所介绍的，你还可以把三条独立语句用分号隔开、串在同一行 (`A = 0; B = 0; C = 0`)。2. 如果你这样赋值：`A = B = C = []`，那么三个名字都引用同一个对象，所以从其中一个名字做原地修改（如 `A.append(99)`）会影响其他名字。这仅对列表和字典这类可变对象的原地修改成立；对数字和字符串这类不可变对象，这个问题不成立，因为它们永远不能被原地修改。3. 列表的 `sort` 方法与 `append` 一样，是对目标列表做原地修改——它返回 `None`，而不是它修改过的列表。把结果赋回 `L`，会让 `L` 变成 `None`，而不是排好序的列表。正如本书前后都讨论过的（如第 8 章），一个较新的内置函数 `sorted` 可以对任何序列排序并返回包含排序结果的新列表；因为它不是原地修改，其结果可以安全而有意义地赋给一个名字。4. 要在单次 `print` 操作中打印到文件，可以使用 `print(X, file=F)` 这种调用形式；或者把 `sys.stdout` 赋给一个手动打开的文件（打印前赋值，需要时打印后恢复原始值）。你也可以用系统 shell 里的特殊语法如 `> file` 把程序的所有打印文本重定向到文件，但这就超出 Python 的范畴了。

11.22 旁注：为什么你会关心——print 与 std out (WHY YOU WILL CARE: PRINT AND S TDOUT)

英文原文

The equivalence between the print operation and writing to sys.stdout makes it possible to reassign sys.stdout to any user-defined object that provides the same write method as files. Because the print statement just sends text to the sys.stdout.write method, you can capture printed text in your programs by assigning sys.stdout to an object whose write method processes the text in arbitrary ways. For instance, you can send printed text to a GUI window, or tee it off to multiple destinations, by defining an object with a write method that does the required routing. You'll see an example of this trick when we study classes in Part VI of this book, but as an abstract preview, it looks like this:

```
class FileFaker:
    def write(self, string):
        # Do something with the printed text in string
import sys
sys.stdout = FileFaker()
print(someObjects)
# Sends text to class write method
```

This works because print is what we will call in the next part of this book a polymorphic operation—it doesn't care what sys.stdout is, only that it has a method (i.e., interface) called write. This redirection to objects is made even simpler with the file key

word argument of print, because we don't need to reset `sys.stdout` explicitly—normal prints will still be routed to the stdout stream: `myobj = FileFaker()` # Redirect to object for one print `print(someObjects, file=myobj)` # Does not reset `sys.stdout` Python's built-in input function reads from the `sys.stdin` file, so you can intercept read requests in a similar way, using classes that implement file-like read methods instead. See the input and while loop example in Chapter 10 for more background on this function. Notice that because printed text goes to the stdout stream, it's also the way to print HTML reply pages in server-side scripts used on the web, and enables you to redirect Python script input and output at the operating system's shell command line as usual: `python script.py < inputfile > outputfile` `python script.py | filterProgram` Python's print operation redirection tools are essentially pure-Python alternatives to some of these shell syntax forms. See other resources for more on web scripts and shell syntax.

中文翻译

print 操作与写 `sys.stdout` 的等价关系，使我们可以把 `sys.stdout` 重新赋值为任何提供了与文件相同 `write` 方法的自定义对象。因为 `print` 语句只是把文本送给 `sys.stdout.write` 方法，你可以通过把 `sys.stdout` 赋给一个 `write` 方法能以任意方式处理文本的对象，从而在程序中捕获打印文本。例如，通过定义一个带 `write` 方法、按需要路由的对象

，你可以把打印文本送到 GUI 窗口，或者"分路" (tee) 到多个目的地。我们在本书第六部分学习类时会在例子里看到这个小花招，但作为抽象预览，它长这样：

```
class FileFaker:
    def write(self, string): # 对 string 里的打印文本做一些处理
import sys
sys.stdout = FileFaker()
print(someObjects) # 把文本送到类的 write 方法它能工作，是因为 print
# 在本书下一部分将被称为多态 (polymorphic) 操作——它不关心 sys.stdout 是什么，只关心它有没有一个叫 write 的方法（即接口）。借助 print 的 file 关键字参数，这种向对象重定向甚至更简单，因为我们不需要显式重置 sys.stdout——正常打印仍然路由到 stdout 流：
myobj = FileFaker() # 为单次打印重定向到对象
print(someObjects, file=myobj) # 不重置 sys.stdout
Python 的内置 input 函数从 sys.stdin 文件读取，所以你可以用类似方式拦截读请求，使用实现文件类 read 方法的类。关于这个函数的更多背景，见第 10 章的 input 和 while 循环示例。注意，因为打印文本流向 stdout 流，它也是服务器端 Web 脚本打印 HTML 回复页面的方式，并让你像往常一样在操作系统 shell 命令行重定向 Python 脚本的输入输出：
python script.py < inputfile > outputfile
python script.py | filterProgram
Python 的 print 操作重定向工具，本质上就是这些 shell 语法形式的纯 Python 替代品。更多 Web 脚本和 shell 语法的内容，请看其他资源。
```

深度理解

·核心概念：多态 (polymorphism) 是这段旁注的关键词——print 只认接口 (write 方法) 不认类型。这正是第 19 章面向对象"鸭子类型"的前奏：替换 stdout 不需要继

承文件类，任何有 write 的对象都行。

·底层实现：print 的 C 代码最终做 PyObjectCallMethod (file, "write", ...)——按名字查方法，不存在就抛 AttributeError。所以自定义对象只要实现 write(self, string) 即可，无需任何注册。

·设计原因：这一设计把"输出目标"变成可插拔的运行时配置，让日志系统、GUI、Web 框架都能无侵入地接管全部输出。

·实际问题：contextlib.redirectstdout 标准库就是这么实现的 (sys.stdout = target, 退出时恢复)；logging.StreamHandler(stream=...) 同理。服务器脚本 print(html) 直接输出网页。

·初学者误区：以为 print 只能写屏幕或文件——它能"写"到任何定义了 write 的对象，包括字符串缓冲 (io.StringIO) ——这是测试代码捕获输出的标准手法。

代码分析

```
import sys, io
capture = io.StringIO()          # "" write
sys.stdout = capture             #
print('hello', 'world')         # capture
sys.stdout = sys.__stdout__      # sys.__stdout__
text = capture.getvalue()        # 'hello world\n'——""

class FileFaker:                 #
    def write(self, string):      # print
        print(f'[faked] {string}', file=sys.__stdout__)
print('boom', file=FileFaker()) #
```

解释器如何处理：`sys.stdout = capture` 只是让模块属性 `sys.stdout` 指向新对象；下一次 `print` 执行时，C 实现重新读取 `sys.stdout` 属性、调用其 `write`。`io.StringIO` 与真实文件都是“文本流协议”的成员，所以完全兼容。这种“按接口替换、运行时查属性”的模式贯穿 Python——事件循环、日志 handler、WSGI 响应对象皆如此设计。

11.23 脚注 (Footnotes)

英文原文

1. C/C++ programmers also take note: although Python now supports statements like `X += Y`, it still does not have C's auto-increment/decrement operators (e.g., `X++`, `--X`). These don't quite map to the Python object model because Python has no notion of in-place changes to immutable objects like numbers. As a preview of the next section, there also are no augmented-named expression forms today; `+= 1` would be close to `++` but thankfully is fiction. That said, we eventually got `+=` and `:=`, so...2. Exception: Alternative implementations of Python such as Jython might allow reserved words to appear as identifiers in some contexts. See Chapter 2 for more on alternative Pythons.
3. If you've used a more restrictive language like C++, you may be interested to know that there is no notion of C++'s `const` declaration in Python; certain objects may be immutable, but names can always be assi

igned. Python also has ways to hide names in classes and modules, but they're not the same as C++'s declarations (if hiding attributes matters to you, see the coverage of X module names in Chapter 25, X class names in Chapter 31, and the Private and Public class decorators example in Chapter 39). 4. Technically, printing uses the equivalent of str in the internal implementation of Python, but the effect is the same. Besides this to-string conversion role, str is also the name of the string data type and can be used to decode Unicode strings from raw bytes with an extra encoding argument, as you'll learn in Chapter 37; this latter role is an advanced usage that you can safely ignore here. 5. You may also be able to use the stdout attribute in the sys module, which refers to the original value sys.stdout had at program startup time. You still need to restore sys.stdout to sys.stdout to go back to this original stream value, though. See the sys module documentation for more details. Also note that sys.stdout and its cohorts may be None in some GUI programs with no console on which to display tests; be sure to check this where it matters.

中文翻译

1. C/C++ 程序员也请注意：虽然 Python 现在支持 $X += Y$ 这样的语句，但它仍然没有 C 语言的自增/自减运算符（如 $X++$ 、 $--X$ ）。这些运算符不能直接映射到 Python 的对象模型，因为 Python 没有“对数字这类不可变对象做原地

修改"的概念。作为下一节的预告：目前也没有增强命名表达式形式；`:= 1` 会接近 `++`，但谢天谢地它只是虚构。不过，我们最终有了 `+=` 和 `:=`，所以.....2. 例外：Jython 这样的 Python 替代实现可能在某些上下文中允许保留字作为标识符出现。更多关于替代性 Python 的内容见第 2 章。3. 如果你用过 C++ 这类更受限的语言，你可能想知道：Python 中没有 C++ 的 `const` 声明概念；某些对象可能是不可变的，但名字总是可以被赋值。Python 也有在类和模块中隐藏名字的方式，但它们不同于 C++ 的声明（如果你在意属性隐藏，见第 25 章 X 模块名、第 31 章 X 类名的讲解，以及第 39 章 Private 与 Public 类装饰器示例）。4. 严格来说，打印在 Python 的内部实现中使用的是 `str` 的等价物，但效果相同。除了这种转字符串的角色，`str` 也是字符串类型的名字，还可以带额外的编码参数把原始字节解码成 Unicode 字符串，你将在第 37 章学到；后一种角色是高级用法，这里可以放心忽略。5. 你也可以使用 `sys` 模块中的 `stdout` 属性，它指向程序启动时 `sys.stdout` 的原始值。不过，你仍然需要把 `sys.stdout` 恢复为 `sys.stdout` 才能回到这个原始流值。更多细节见 `sys` 模块文档。另外注意，在某些没有控制台可显示文本的 GUI 程序中，`sys.stdout` 及其同侪可能是 `None`；在需要的地方务必检查。

本章总结

深度理解

·核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。

- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

技术拓展 (Technical Expansion)

实际项目中的应用场景

- 数据解析：`first, rest = row` 拆 CSV/日志行，`body, last = tokens` 取句尾；嵌套解包从 JSON 式嵌套结构取已知字段。
- 初始化与配置：`width = height = margin = 0` 一次初始化；`red, green, blue = range(3)` 枚举色值；但可变默认值 (`a = b = []`) 是经典反模式，必记。
- 循环与迭代：`while (line := f.readline())`: 读文件；`for (a, b, c) in data`: 变长解包；`x += [item]` 累积列表 (`extend` 语义)。
- 日志与输出：`print(..., file=sys.stderr)` 错误输出；`sys.stdout` 全局重定向到日志文件；`io.StringIO` 捕获输出做测试断言；`print(..., end="", flush=True)` 做进度指示。
- 性能：`+=` 的原地路径让 `for item in big: totallist += [item]` 远快于 `totallist = totallist + [item]` (后者每次 $O(n)$)

n) 拷贝)。

与其他语言的区别

语言	赋值/输出模型	与 Python 的差异
C/C++	赋值是“存值”；有 ++、--；printf 是库函数	C 里 a = b = c 是表达式（可嵌套）；无解包、无 =args；++ 在 Python 中不存在
Java	变量有静态类型；System.out.println 方法调用	Java 无多重目标赋值、无序列解包；int x = 5 声明与赋值分开
C++	有 const 声明约束名字	Python 无 const：对象可不可变由对象决定，名字永远可重新绑定
Python	赋值=绑定引用；名字无类型	全部一致性来自“引用语义 + 协议（接口）”：解包、重定向、多态输出都建立其上

Python 发展历史背景

- 赋值解包：Python 1.x 时代仅有元组/列表解包（“1,2 = a,b”变体），扩展解包由 PEP 3132 在 3.0 引入；字面量全面展开由 PEP 448 在 3.5 引入。
- print 语句 → 函数：PEP 3105 (3.0) ——本书开篇就说的“3.X 破坏了几几乎所有 2.X 程序”正是这一条。sep/end/file/flush 也因此得以存在。
- 海象运算符 :=：PEP 572 (3.8)，当时引发社区激烈辩论（与“显式优于隐式”哲学的张力）；PEP 自带了“不要过度使用”的忠告，本书作者态度与之一致。
- 软保留字 match/case/type：PEP 634 (3.10 match 语句)、PEP 695 (3.12 type 语句) ——用上下文相关语法避免破坏老代码。

高级开发者需要掌握的相关知识

- 字节码层面：UNPACKSEQUENCE/UNPACKEX、DUPTOP、INPLACE、STORESUBSCR 的语义（可用 dis.dis 验证）；
- 可变对象共享引用的边界：+= vs +、copy.copy vs copy.deepcopy 的选择；
- sys.stdout/sys.stderr/sys.stdout 及 contextlib.redirectstdout 的用法与陷阱（GUI 程序里可能是 None）；
- := 在推导式中的作用域语义（PEP 572 的特别规则：绑定目标在推导式外也可见）与正则匹配、while 循环中的惯用写法；
- 理解“名字/对象分离”是理解闭包、装饰器、nonlocal、参数传递（第 18、19 章）的前提。

学习建议 (Learning Advice)

- 重要程度：★★★★★ (5/5)。本章是 Python 语法地基中的地基：你写的每一行代码几乎都包含赋值或 print。
- 应该掌握到什么程度：
- 必会（熟练到肌肉记忆）：基本赋值、元组解包与交换、a, rest = ...、多重目标赋值、+= 类增强赋值、print 四个关键字参数、file= 重定向、命名规则与保留字清单、L = L.sort() 这类陷阱的识别。
- 进阶（能看懂并少量使用）：嵌套解包、在函数/字面量/match 中的其他角色、sys.stdout 全局重定向与恢复、:= 在 while/if/推导式中的惯用写法。

- 不必深究：名字注解 (`x: int = 3`) 的细节、Unicode 变量名的完整规则——需要时再查。
- 后续应该学习哪些相关内容：
- 第 12 章 if 与布尔表达式——`:=`、表达式语句在那里配合使用；
- 第 13 章 for/while——循环头解包 (`for (a, b, c) in ...`)、文件读行模式；
- 第 17 章作用域——理解“名字在哪里可见”（本章 11.14 的伏笔）；
- 第 18 章参数传递——args/kwargs 与本章的星号语义同源；
- 第 19 章 OOP——self 与属性赋值目标、init 里的绑定。

本章完（基于《Learning Python》第 6 版第 11 章完整翻译整理）